

卷积神经网络的训练与架构设计

两大主题

如何构建卷积神经网络?

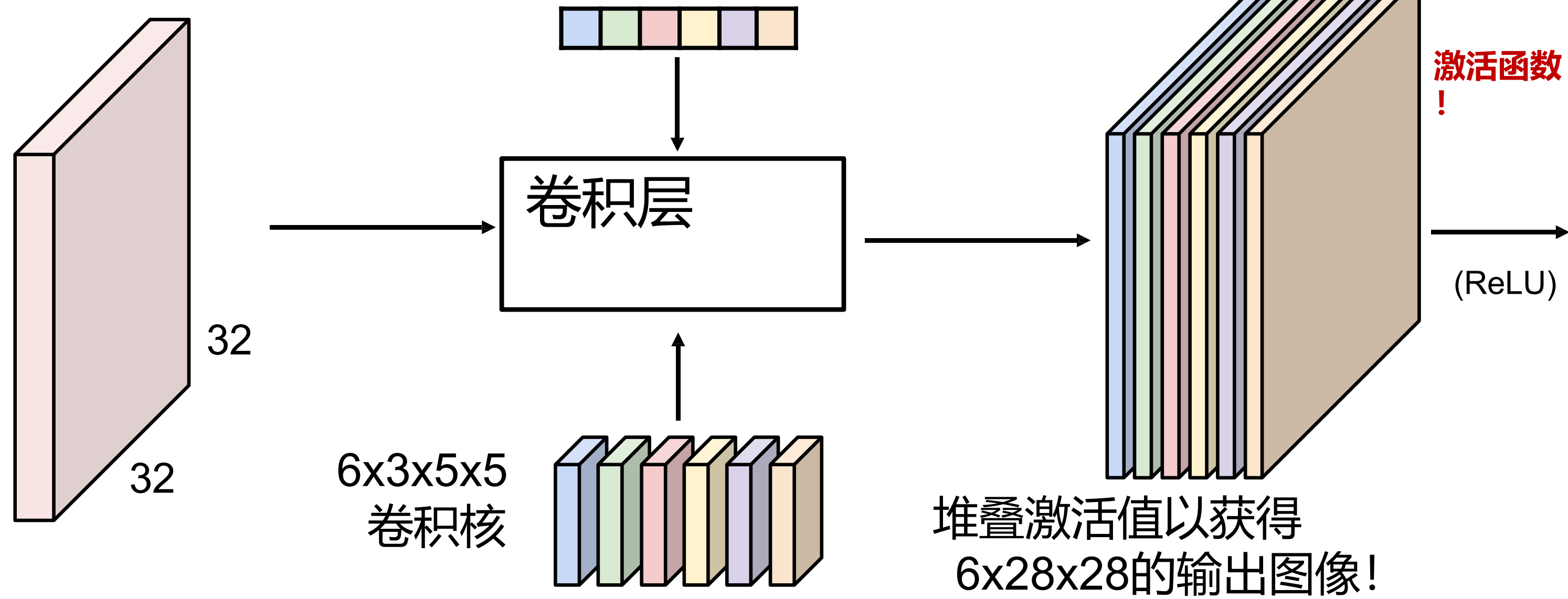
卷积神经网络中的层结构
激活函数
卷积神经网络架构
权重初始化

如何训练卷积神经网络?

数据预处理
数据增强
迁移学习
超参数选择

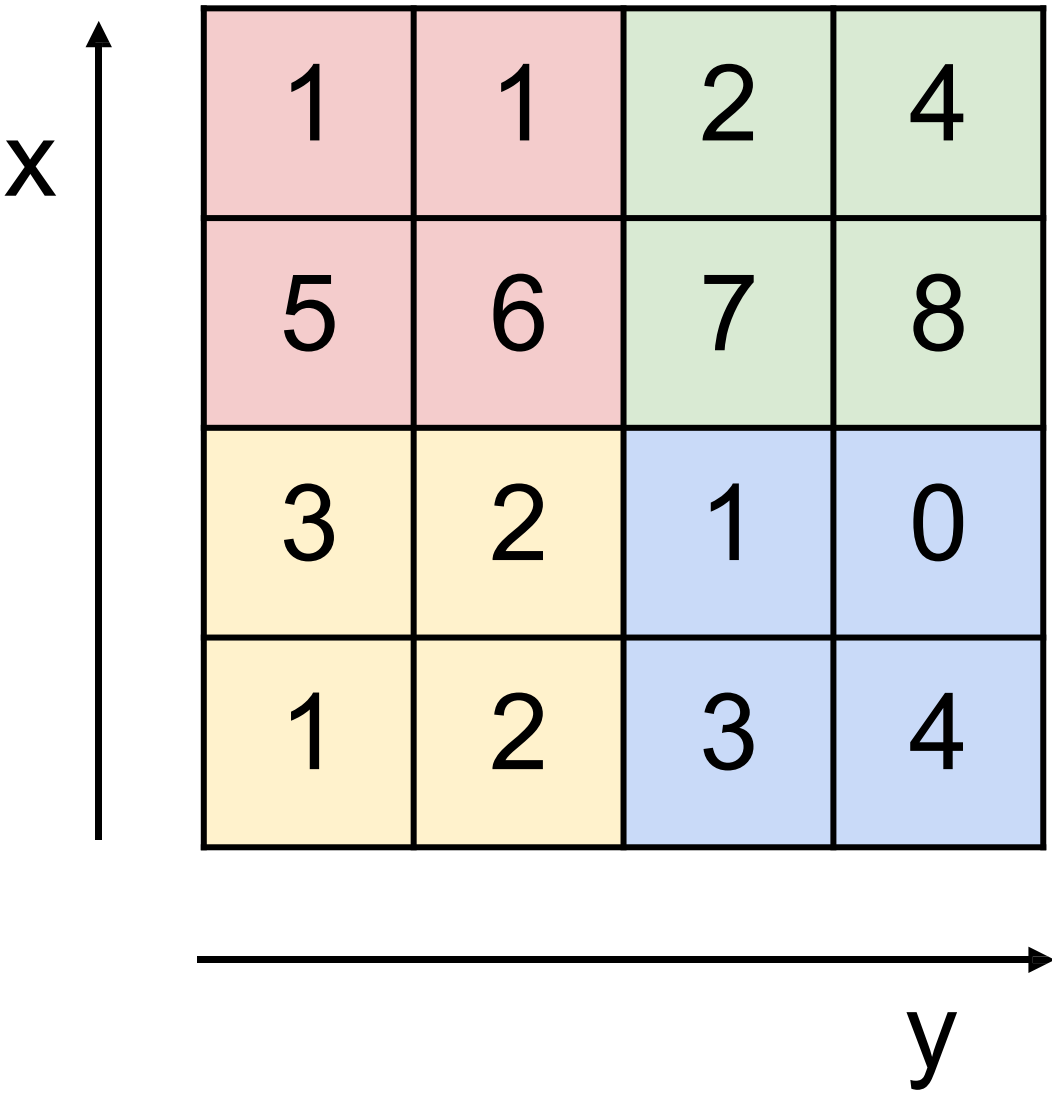
回顾：卷积层

3x32x32 图像 别忘了偏置项！

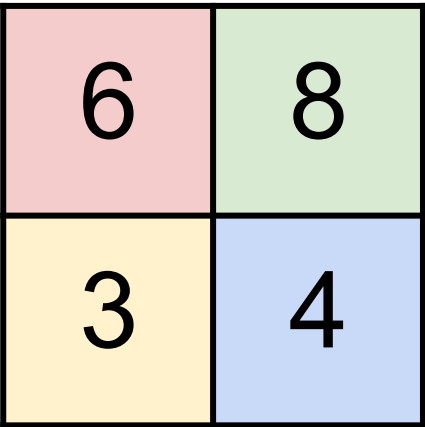


回顾：池化层

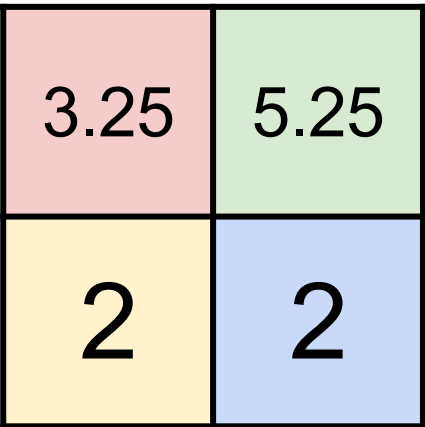
单层深度切片



池化，采用2x2卷积核，步长为2



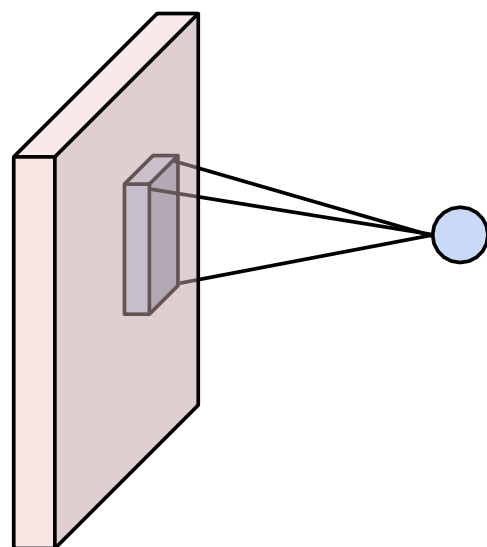
最大池化



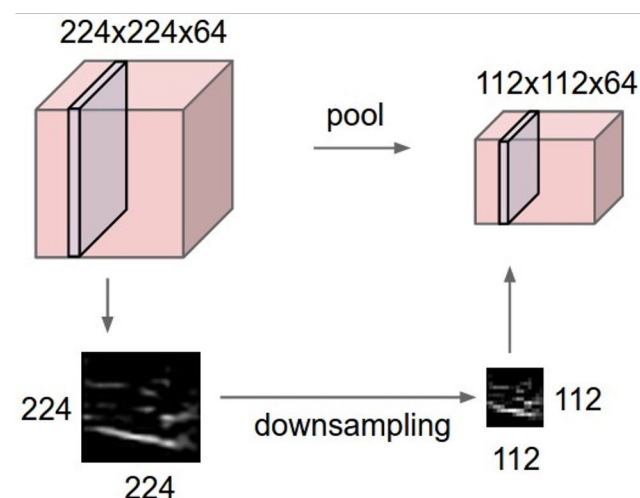
平均池化

卷积神经网络的组成部分

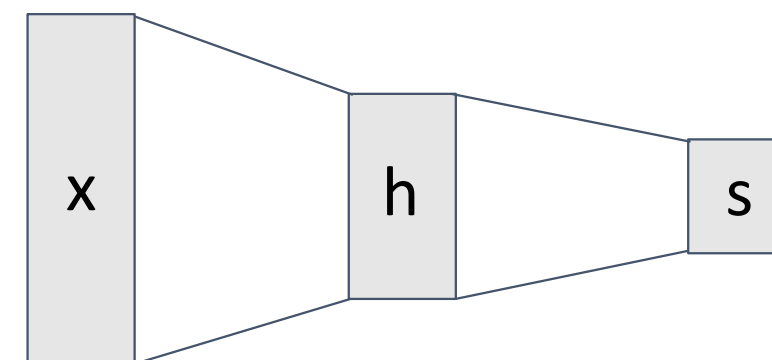
卷积层



池化层



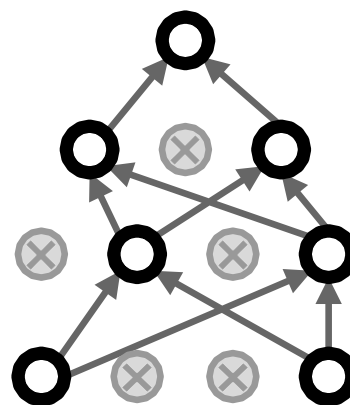
全连接层



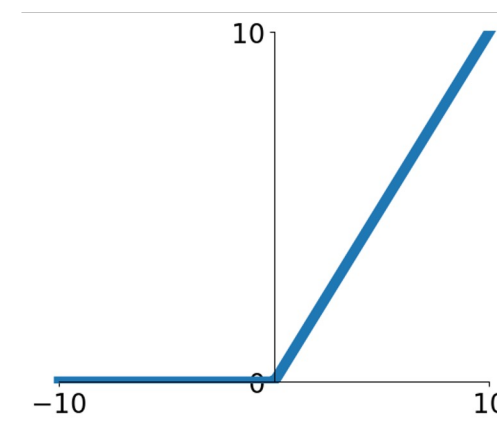
归一化层

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Dropout (有时)



激活函数



示例归一化层：LayerNorm

核心思想：学习可对输入数据进行缩放/平移的参数

1. 对输入数据进行归一化
2. 使用学习到的参数进行缩放/平移

示例归一化层：LayerNorm

核心思想：学习能够对输入数据进行缩放/平移的参数

1. 对输入数据进行归一化
2. 使用学习到的参数进行缩放/偏移

$$\mathbf{x} : \mathbf{N} \times \mathbf{D}$$

归一化

$$\boldsymbol{\mu}, \boldsymbol{\sigma} : \mathbf{N} \times \mathbf{1}$$

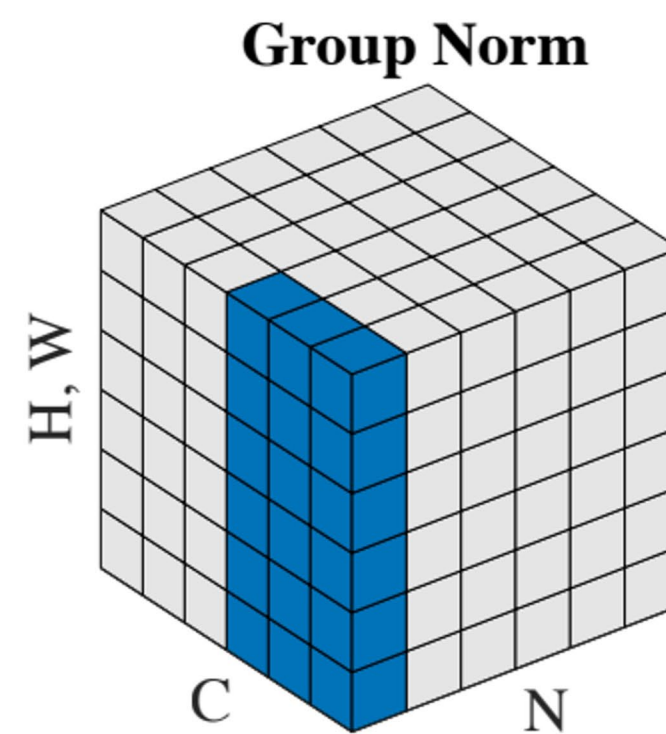
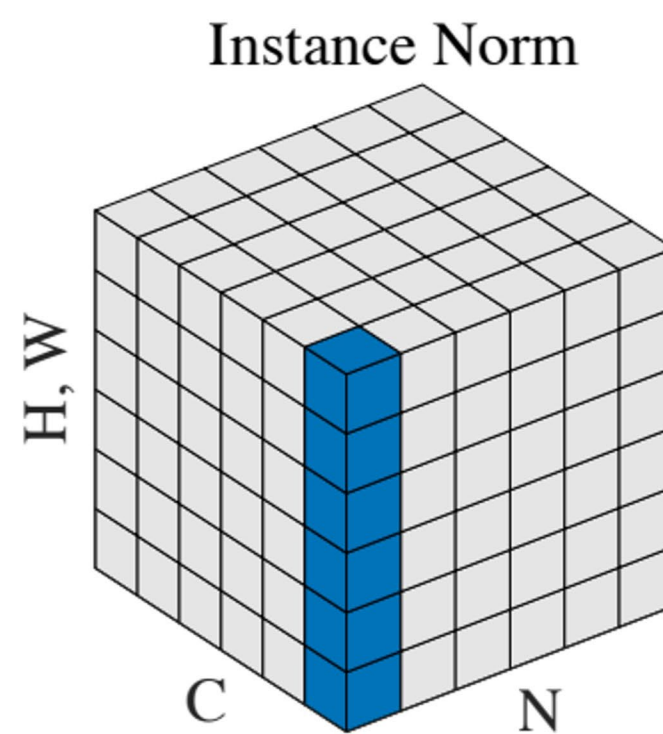
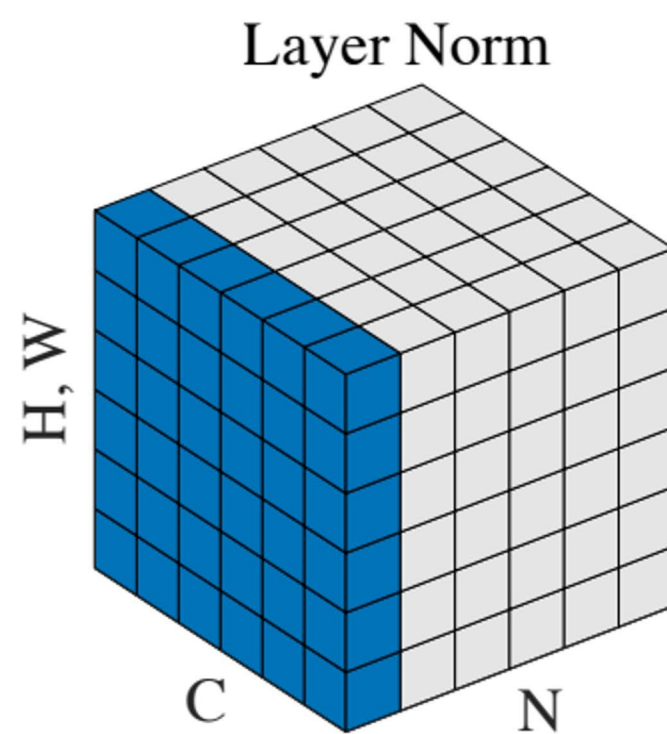
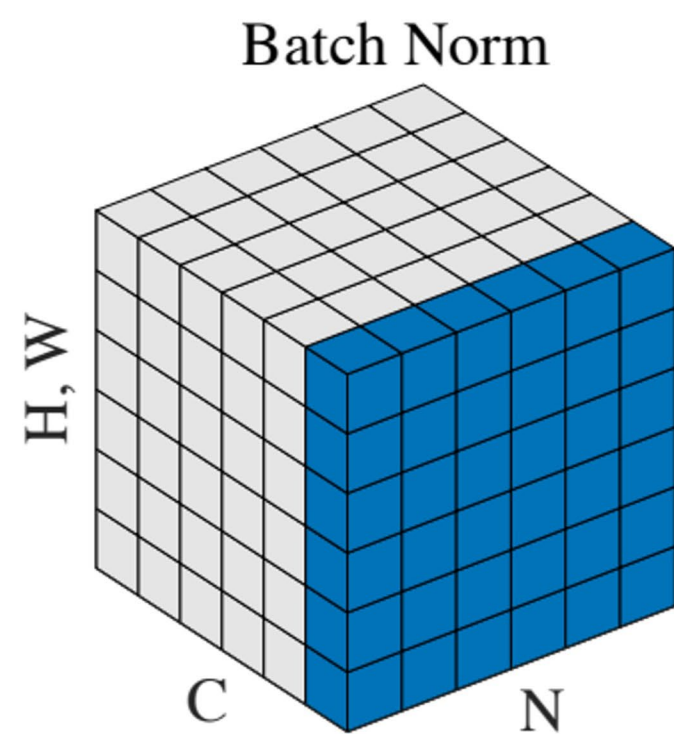
$$\boldsymbol{\gamma}, \boldsymbol{\beta} : \mathbf{1} \times \mathbf{D}$$

$$\mathbf{y} = \boldsymbol{\gamma} (\mathbf{x} - \boldsymbol{\mu}) / \boldsymbol{\sigma} + \boldsymbol{\beta}$$

每批次计算的统计量 →

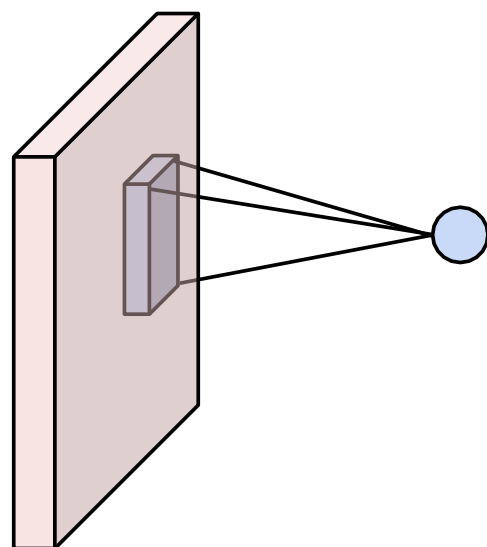
学习到的参数应用于每个样本 →

其他归一化层

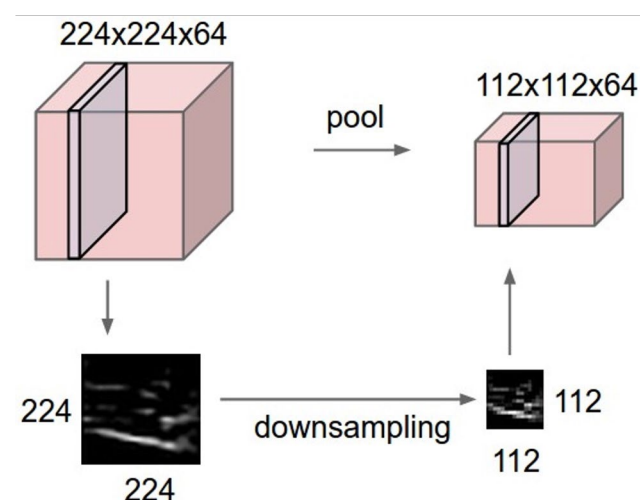


卷积神经网络的组成部分

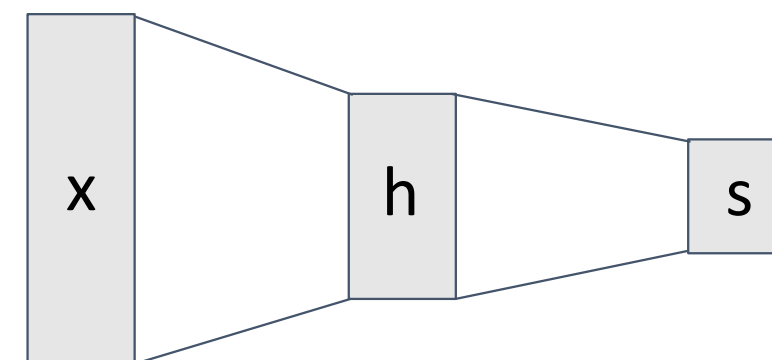
卷积层



池化层



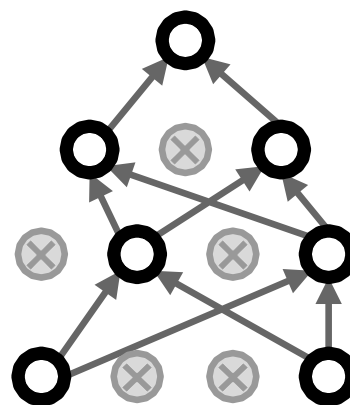
全连接层



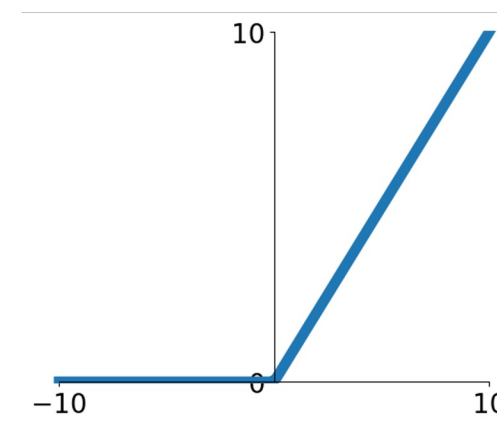
归一化层

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Dropout (有时)

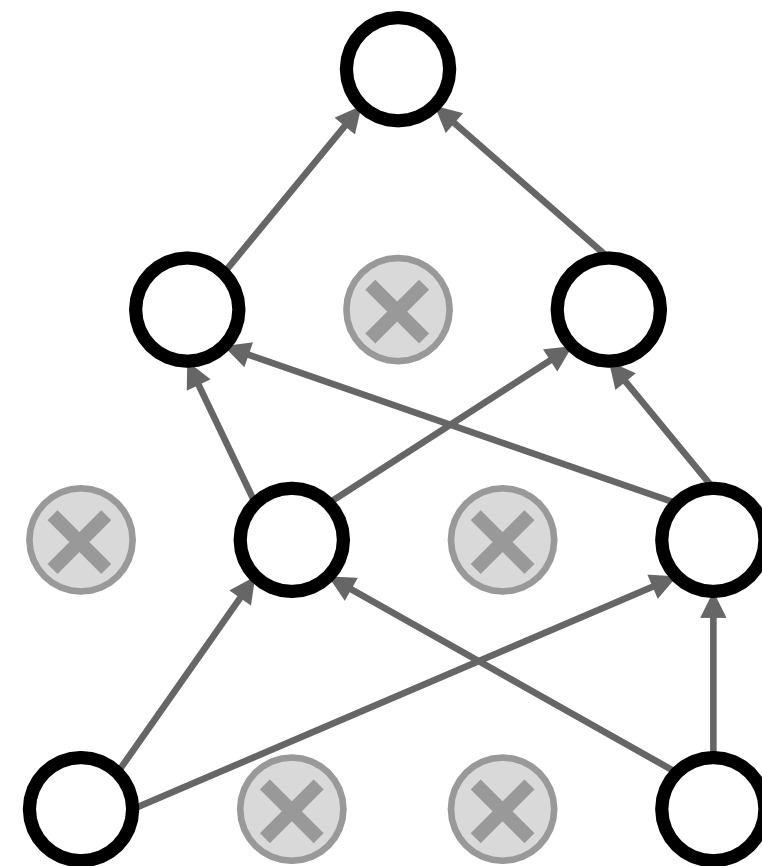
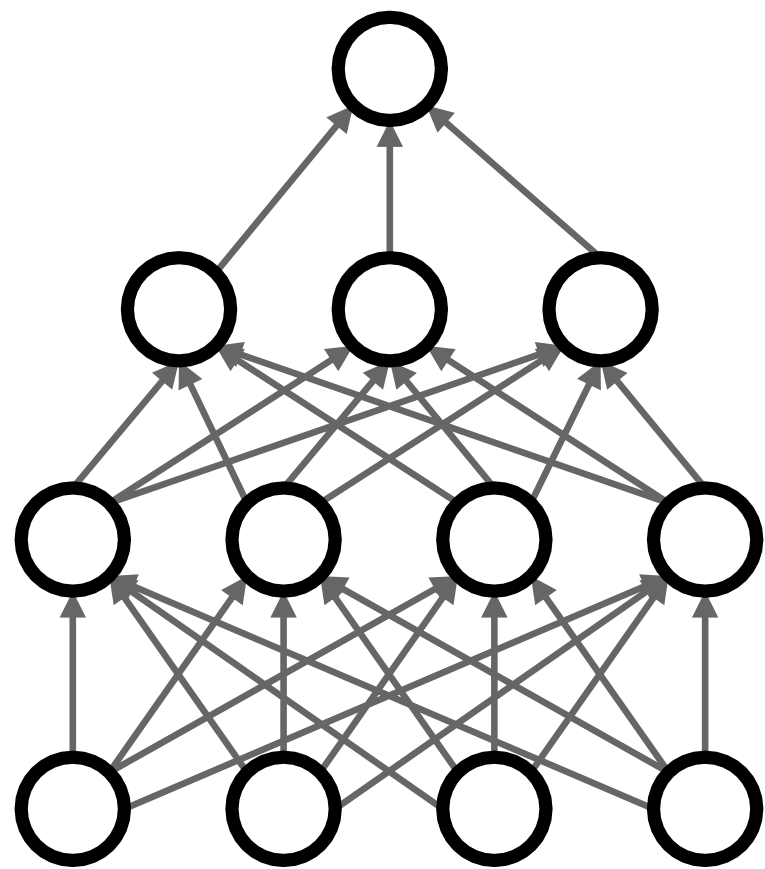


激活函数



正则化：Dropout

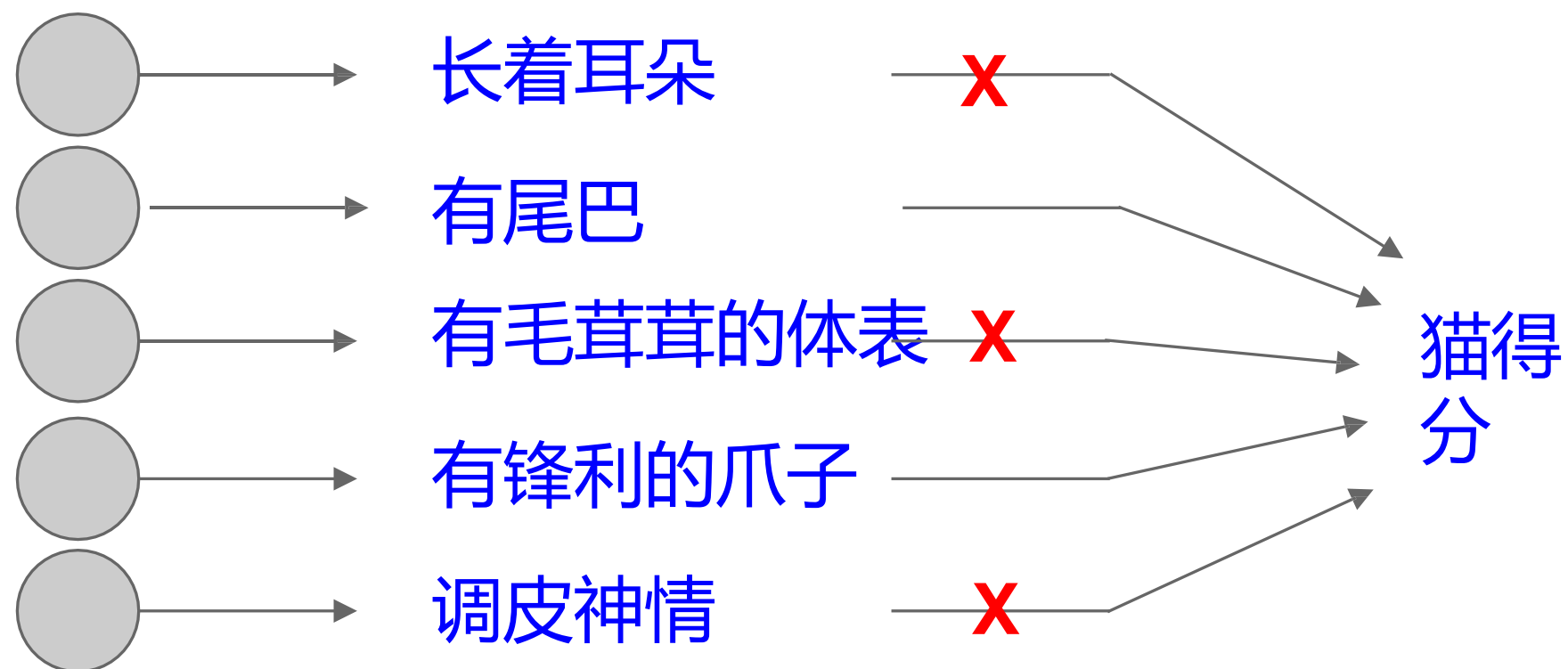
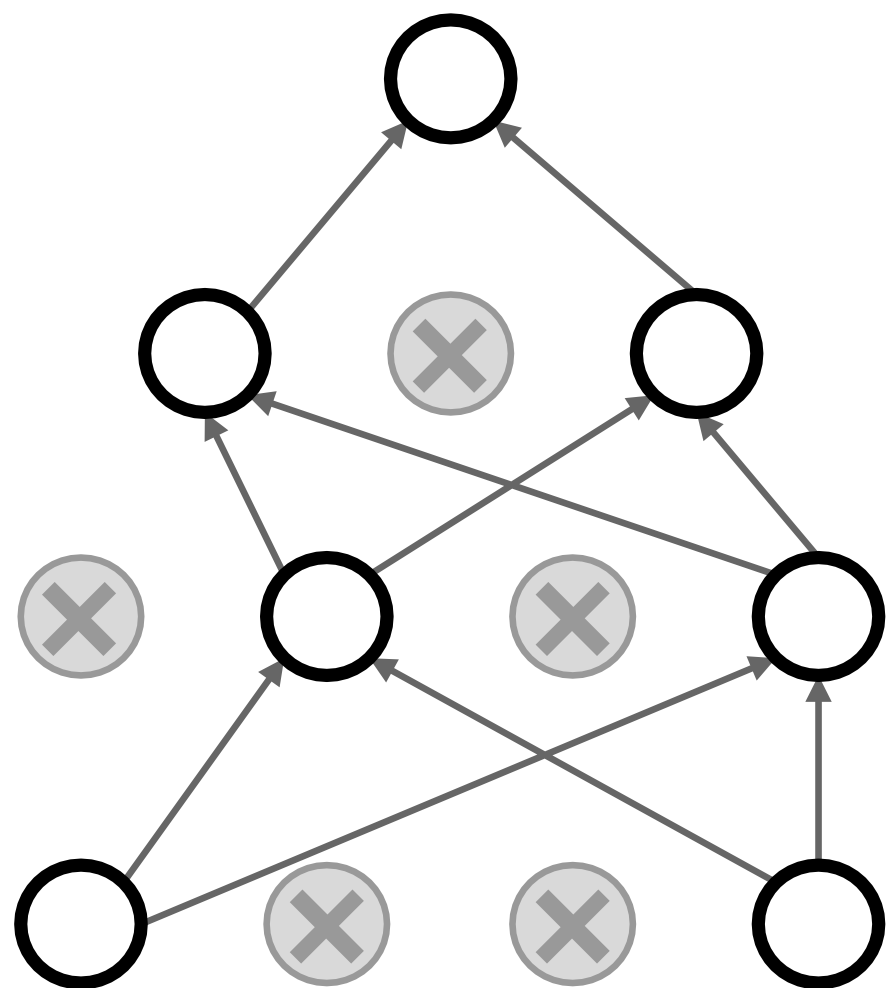
在每次前向传播中，随机将部分神经元置零。丢失概率是超参数，通常取值为0.5。



正则化：Dropout

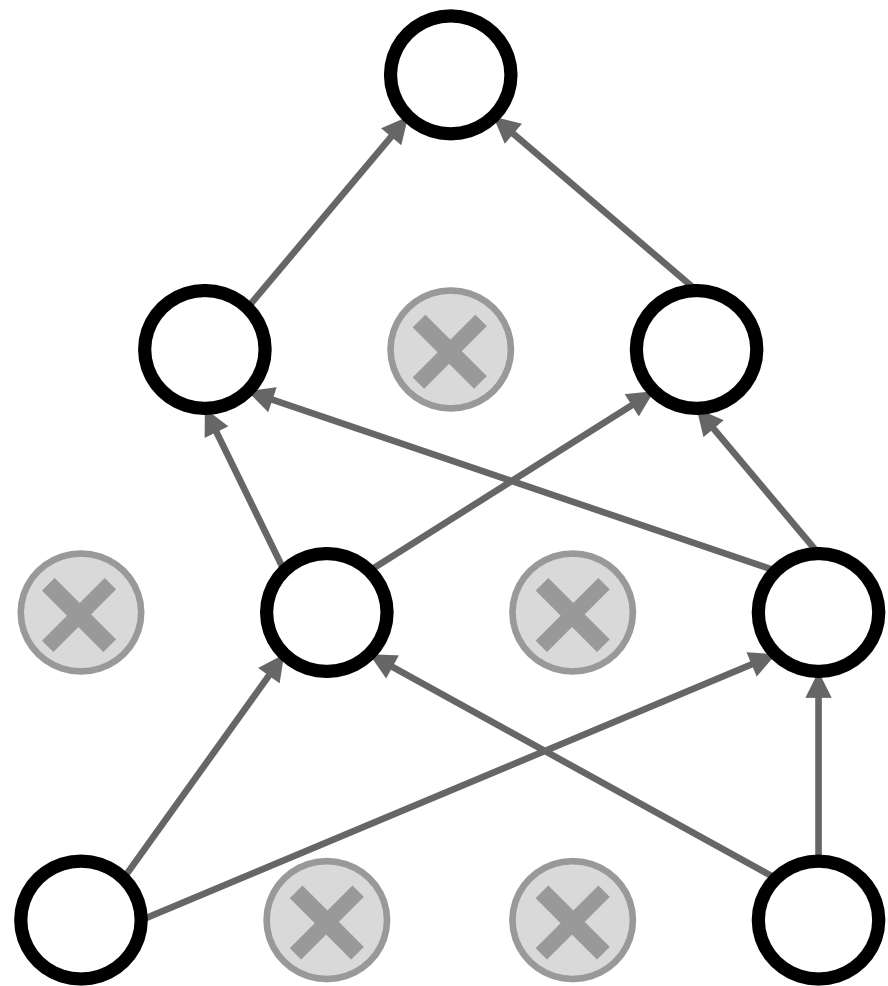
这怎么可能是一个好主意？

迫使网络采用冗余表示；阻碍特征间的协同适应



正则化：Dropout

这怎么可能是一个好主意？



另一种解释：

Dropout本质上是在训练一个共享参数的大型模型**集合**。

每次binary mask即为一个模型

一个含4096个单元的全连接层可生成
 $2^{4096} \approx 10^{1233}$ 种可能的掩码！
而宇宙中仅存在约 10^{82} 个原子...

Dropout: 测试时

```
def predict(X):  
    # ensembled forward pass  
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations  
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations  
    out = np.dot(W3, H2) + b3
```

测试阶段所有神经元始终处于激活状态

→ 我们必须调整激活值，使每个神经元满足：测试时输出
= 训练时预期输出

Dropout 概述

```
""" Vanilla Dropout: Not recommended implementation (see notes below) """
```

```
p = 0.5 # probability of keeping a unit active. higher = less dropout
```

```
def train_step(X):
```

```
    """ X contains the data """
```

```
    # forward pass for example 3-layer neural network
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1)
```

```
    U1 = np.random.rand(*H1.shape) < p # first dropout mask
```

```
    H1 *= U1 # drop!
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2)
```

```
    U2 = np.random.rand(*H2.shape) < p # second dropout mask
```

```
    H2 *= U2 # drop!
```

```
    out = np.dot(W3, H2) + b3
```

```
    # backward pass: compute gradients... (not shown)
```

```
    # perform parameter update... (not shown)
```

```
def predict(X):
```

```
    # ensembled forward pass
```

```
    H1 = np.maximum(0, np.dot(W1, X) + b1) * p # NOTE: scale the activations
```

```
    H2 = np.maximum(0, np.dot(W2, H1) + b2) * p # NOTE: scale the activations
```

```
    out = np.dot(W3, H2) + b3
```

训练时间下降

测试时缩放

两大主题

如何构建卷积神经网络?

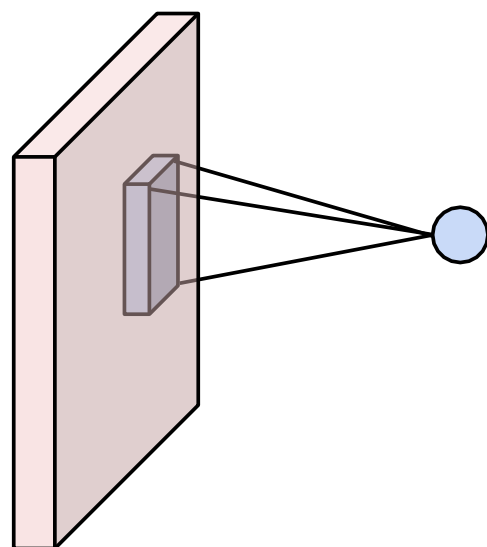
卷积神经网络中的层结构
激活函数
卷积神经网络架构
权重初始化

如何训练卷积神经网络?

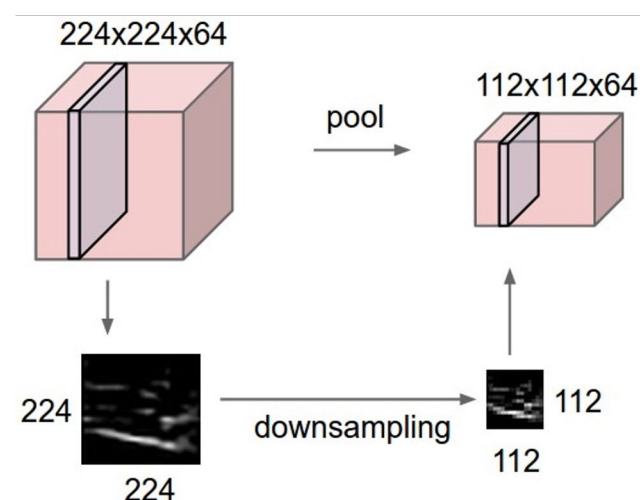
数据预处理
数据增强
迁移学习
超参数选择

卷积神经网络的组成部分

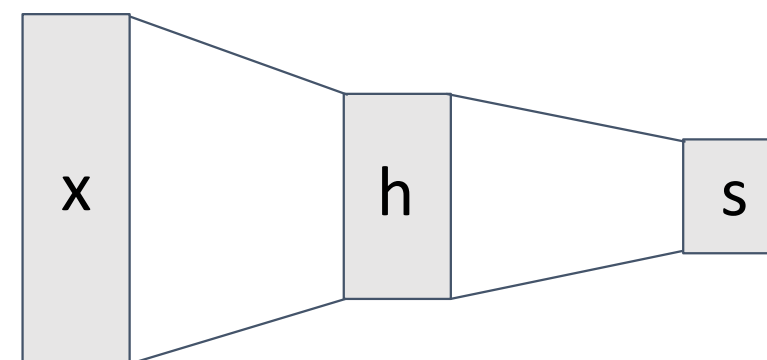
卷积层



池化层



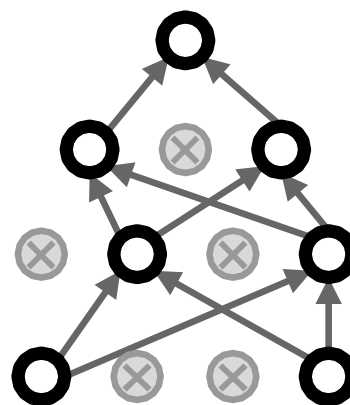
全连接层



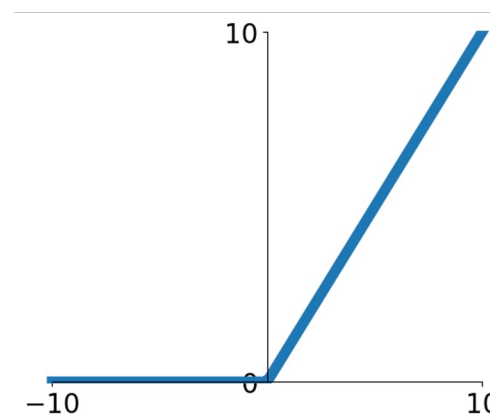
归一化层

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Dropout (有时)

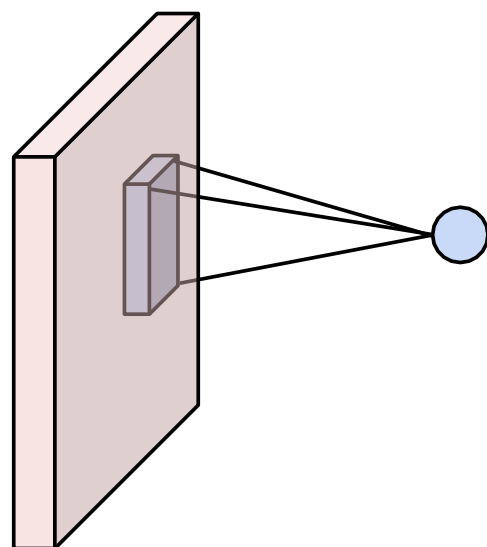


激活函数

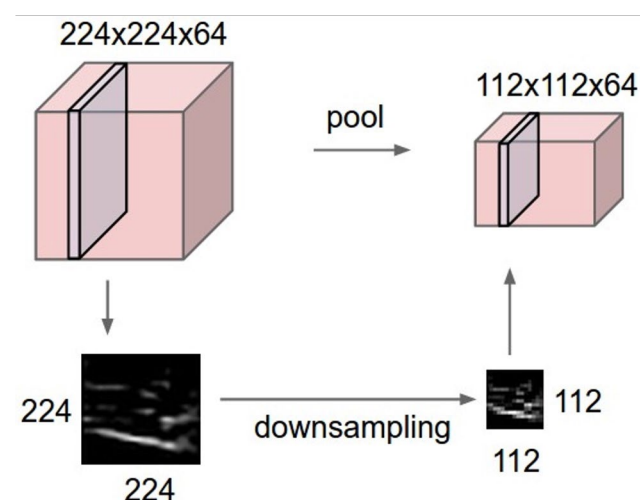


卷积神经网络的组成部分

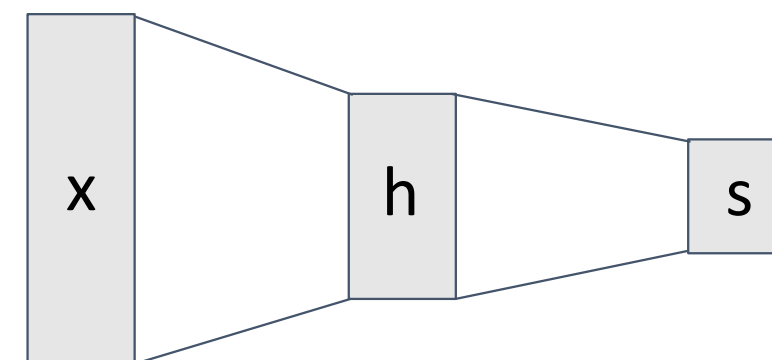
卷积层



池化层

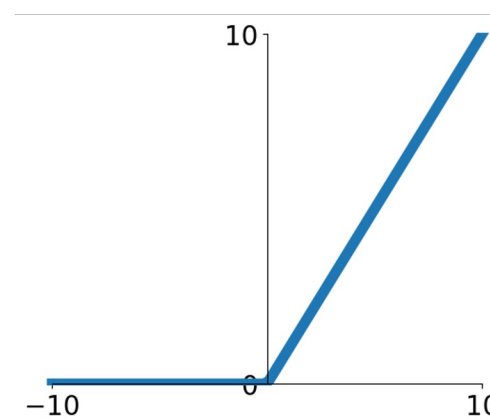


全连接层

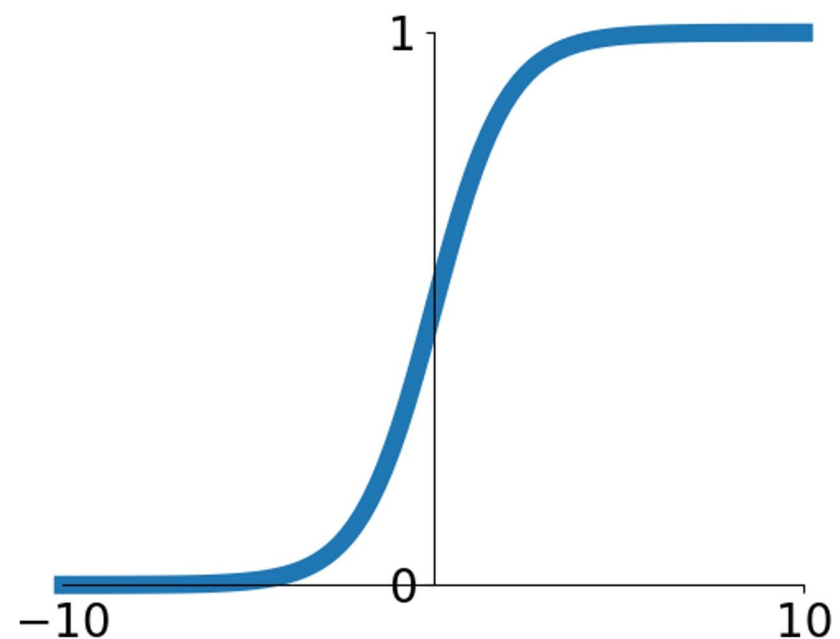


目标：为模型引入**非线性**！

激活函数



激活函数



Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

- 将数值压缩到[0,1]区间
- 历史上广受欢迎，因其可被解释为神经元的饱和"放电率"

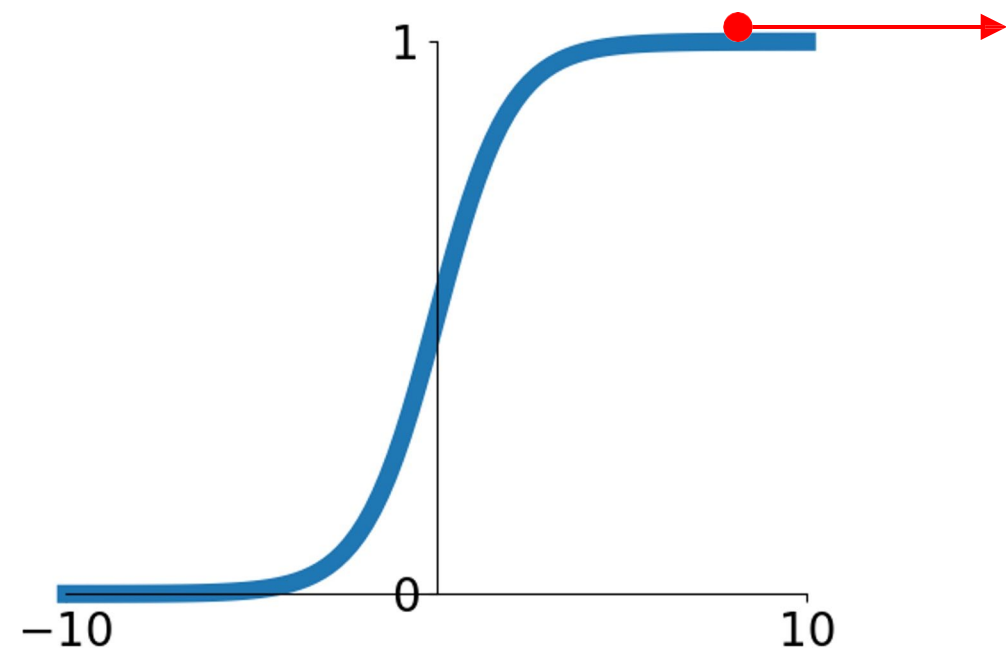
核心问题：

多层sigmoid函数→梯度逐渐减小

问题：在哪些区域中 sigmoid 函数具有较小的梯度？

靠近输入层的隐藏层

激活函数



Sigmoid

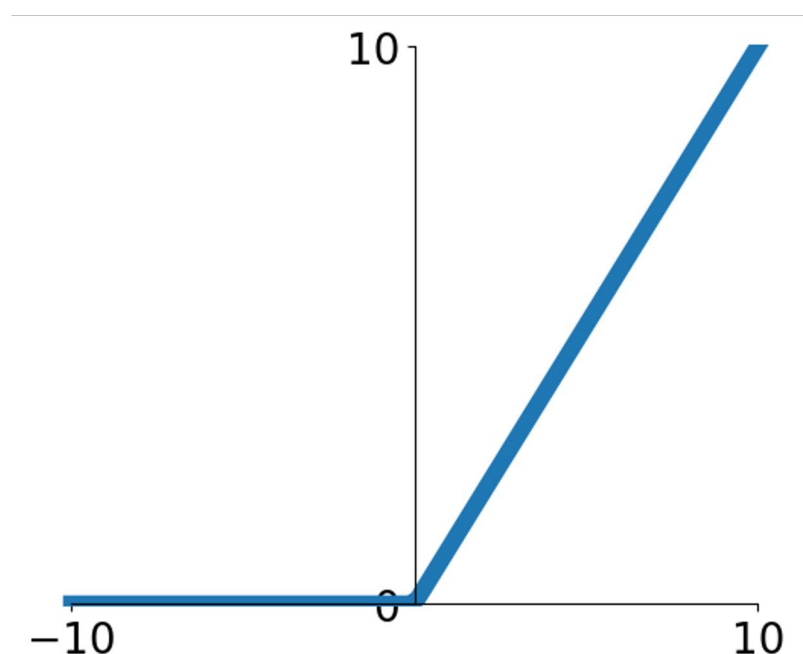
$$\sigma(x) = 1 / (1 + e^{-x})$$

- 将数值压缩到[0,1]区间
- 历史上广受欢迎，因其可被解释为神经元的饱和"放电率"

核心问题：

过大的正负值会导致梯度消失。多层sigmoid组合→实际中梯度持续衰减

激活函数

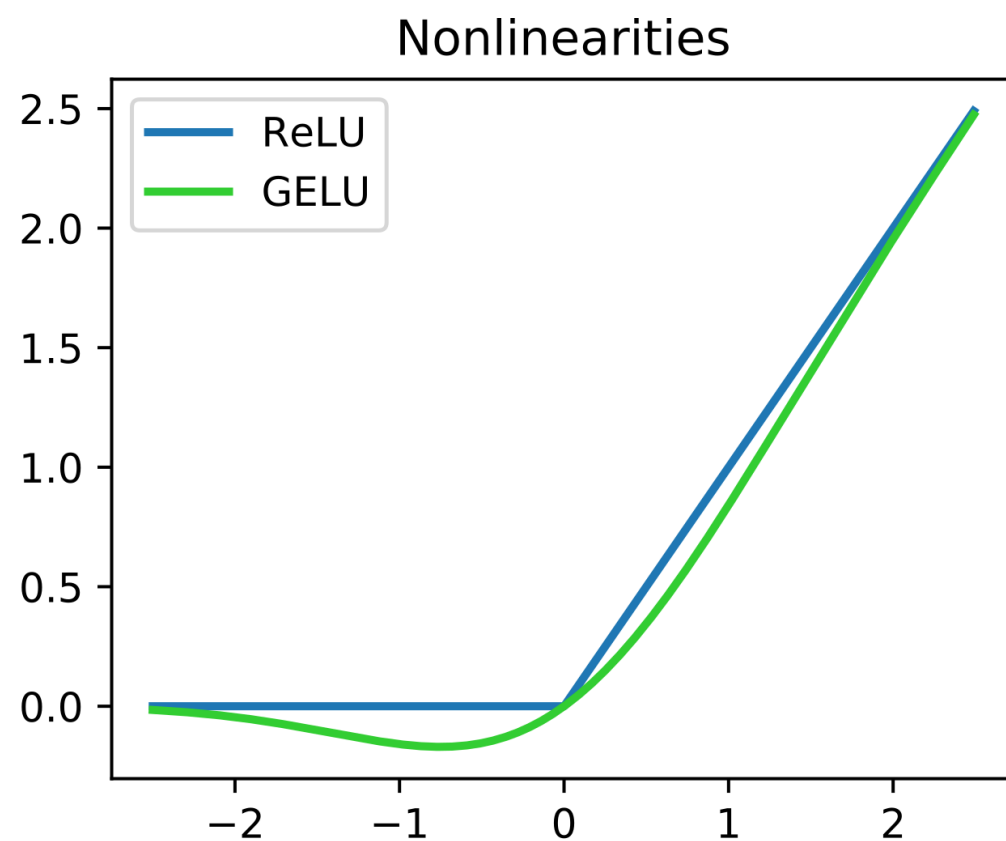


ReLU (整流线性单元)

- 计算 $f(x) = \max(0, x)$
- 在正区域内不饱和
- 计算效率极高
- 实际收敛速度远快于sigmoid (例如快6倍)
- 输出值非零中心化
- 一个烦人的问题:

当输入小于0时会出现死ReLU现象!

激活函数



来源: https://en.m.wikipedia.org/wiki/File:ReLU_and_GELU.svg

- 计算 $f(x) = x * \Phi(x)$

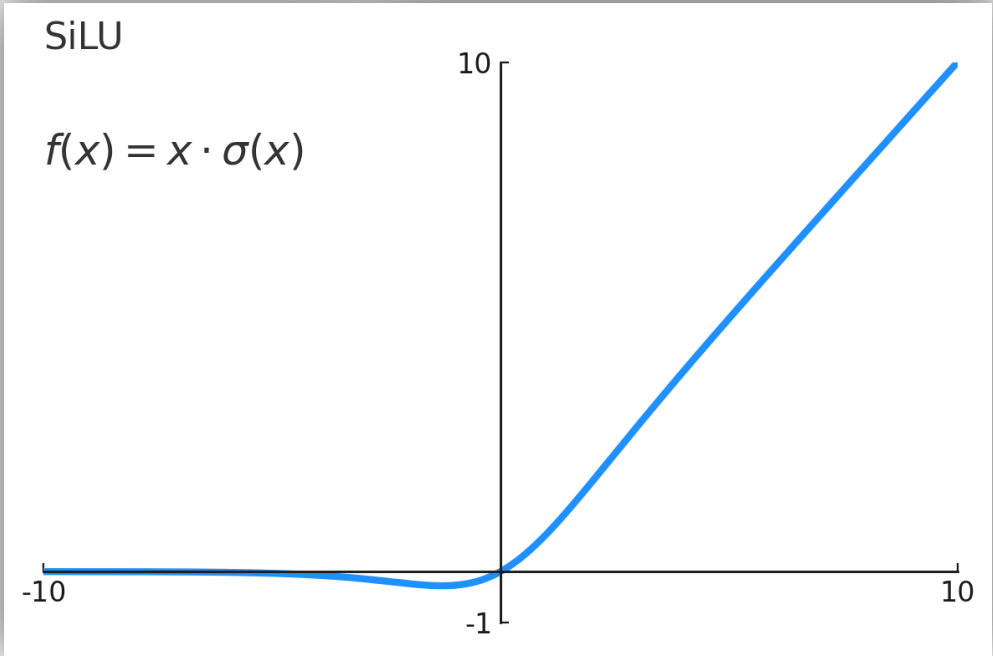
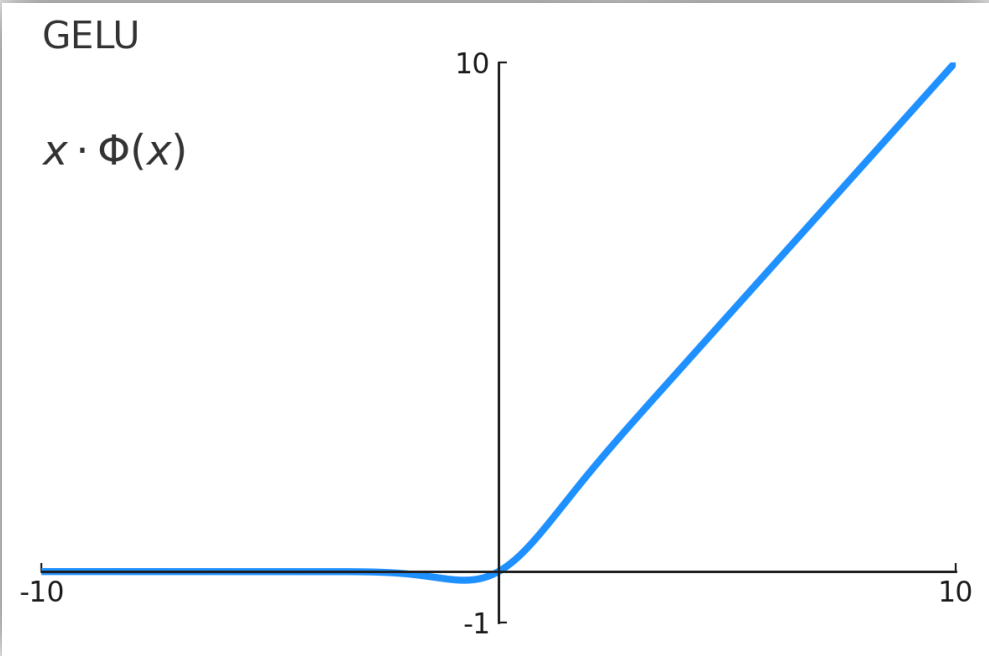
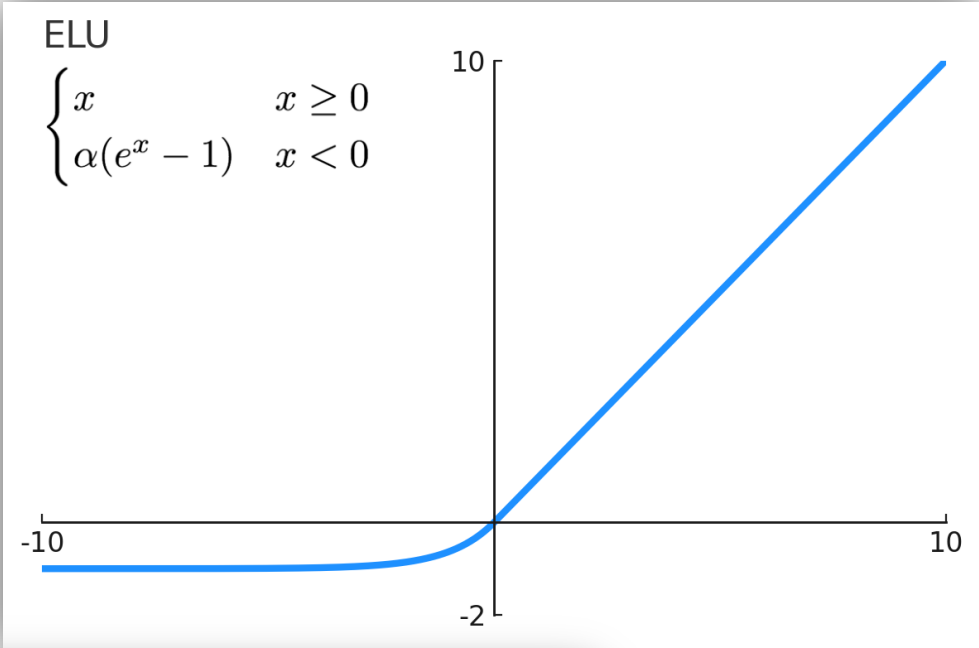
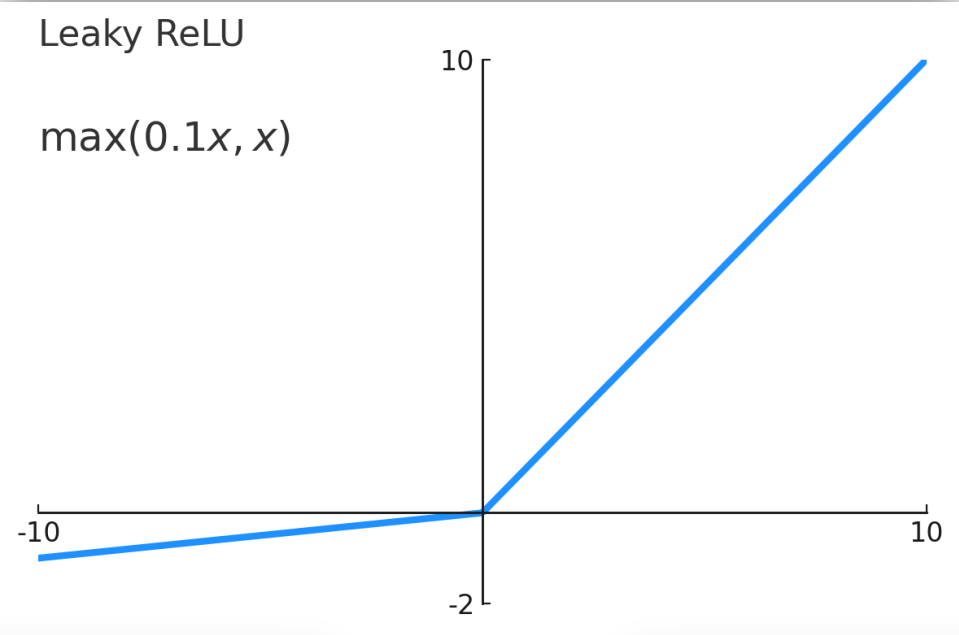
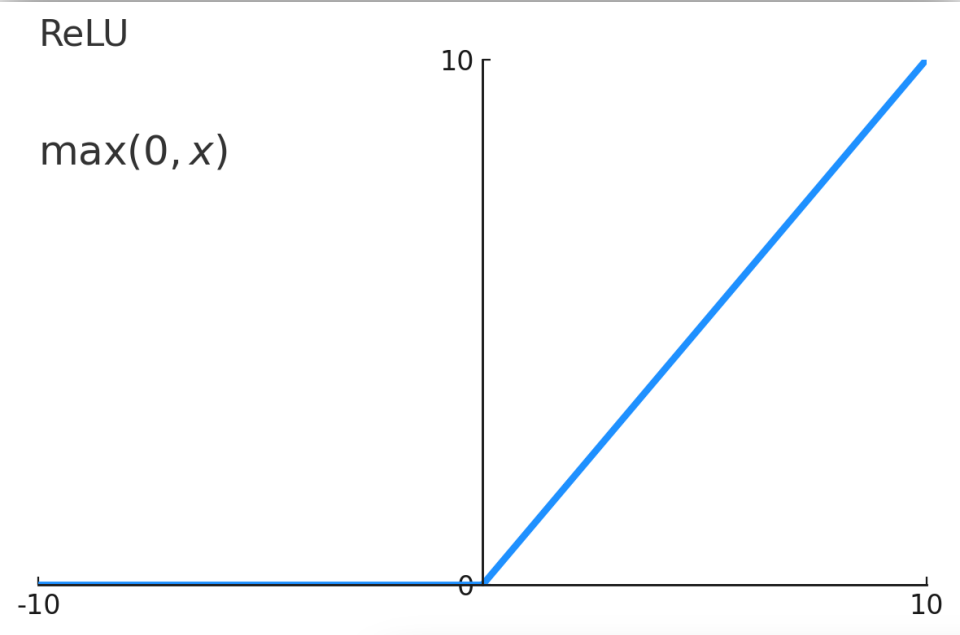
- 在0附近表现优异
- 平滑特性有助于实际训练过程

- 计算成本高于ReLU
- 大负值仍可能导致梯度趋近于零

GELU

(高斯误差线性
单元)

激活函数动物园



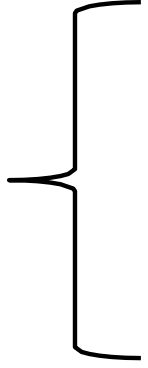
激活函数

问：卷积神经网络中激活函数应用于何处？

答：通常置于线性层之后（如前馈/线性层、卷积层等）

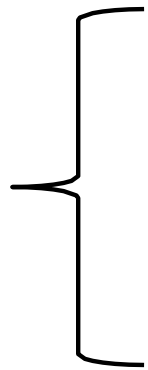
两大主题

如何构建卷积神经网络?



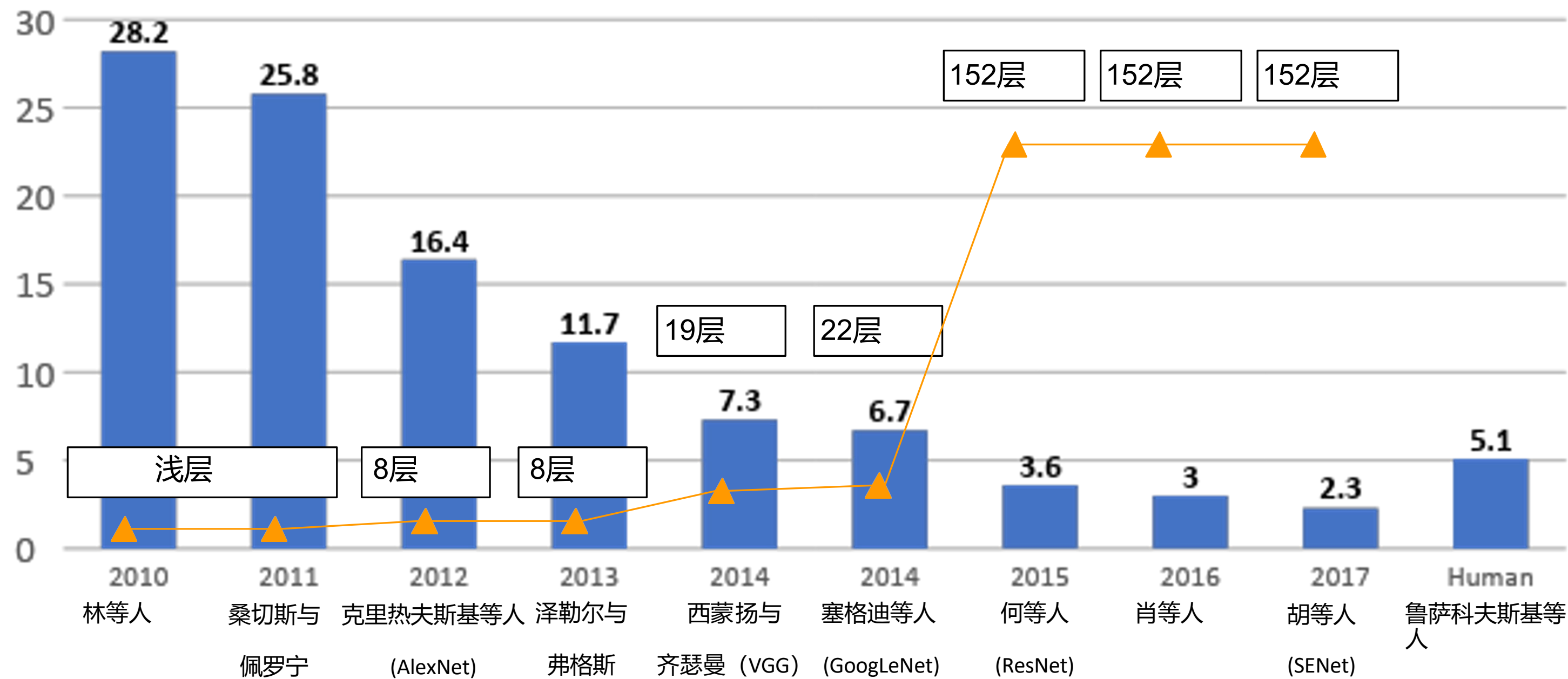
卷积神经网络中的层结构
激活函数
卷积神经网络架构
权重初始化

如何训练卷积神经网络?

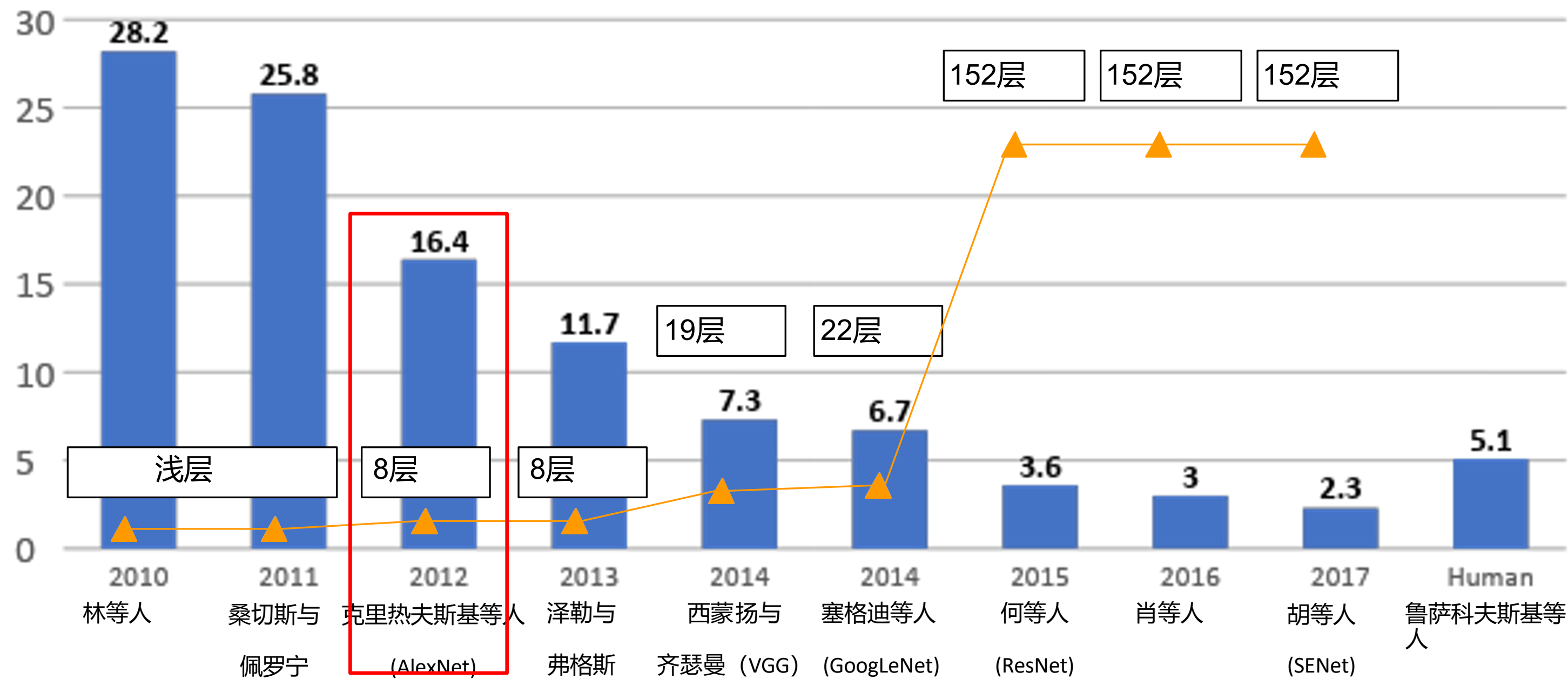


数据预处理
数据增强
迁移学习
超参数选择

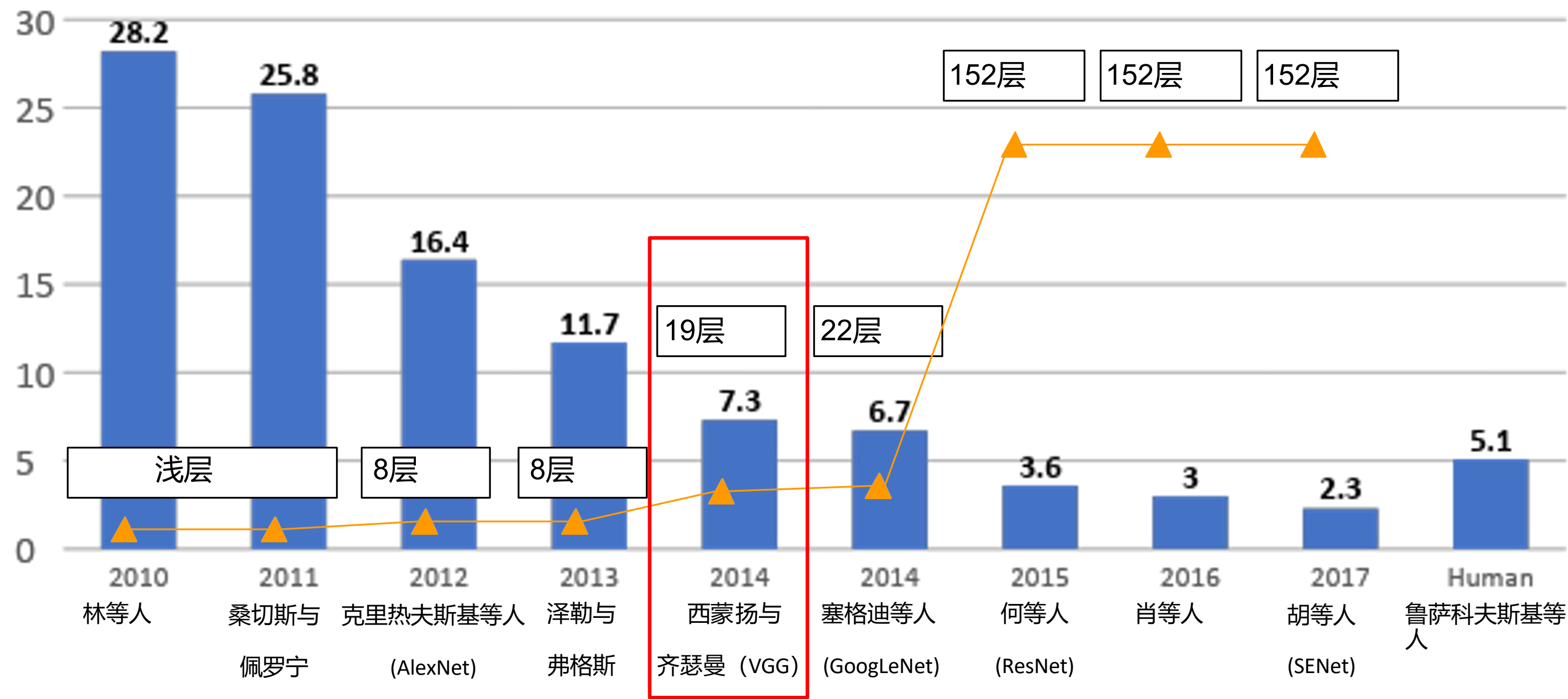
ImageNet大规模视觉识别挑战赛（ILSVRC）获奖者



ImageNet大规模视觉识别挑战赛（ILSVRC）获奖者



ImageNet大规模视觉识别挑战赛（ILSVRC）获奖者



案例研究：VGGNet

小型卷积核，更深层网络

8层 (AlexNet)

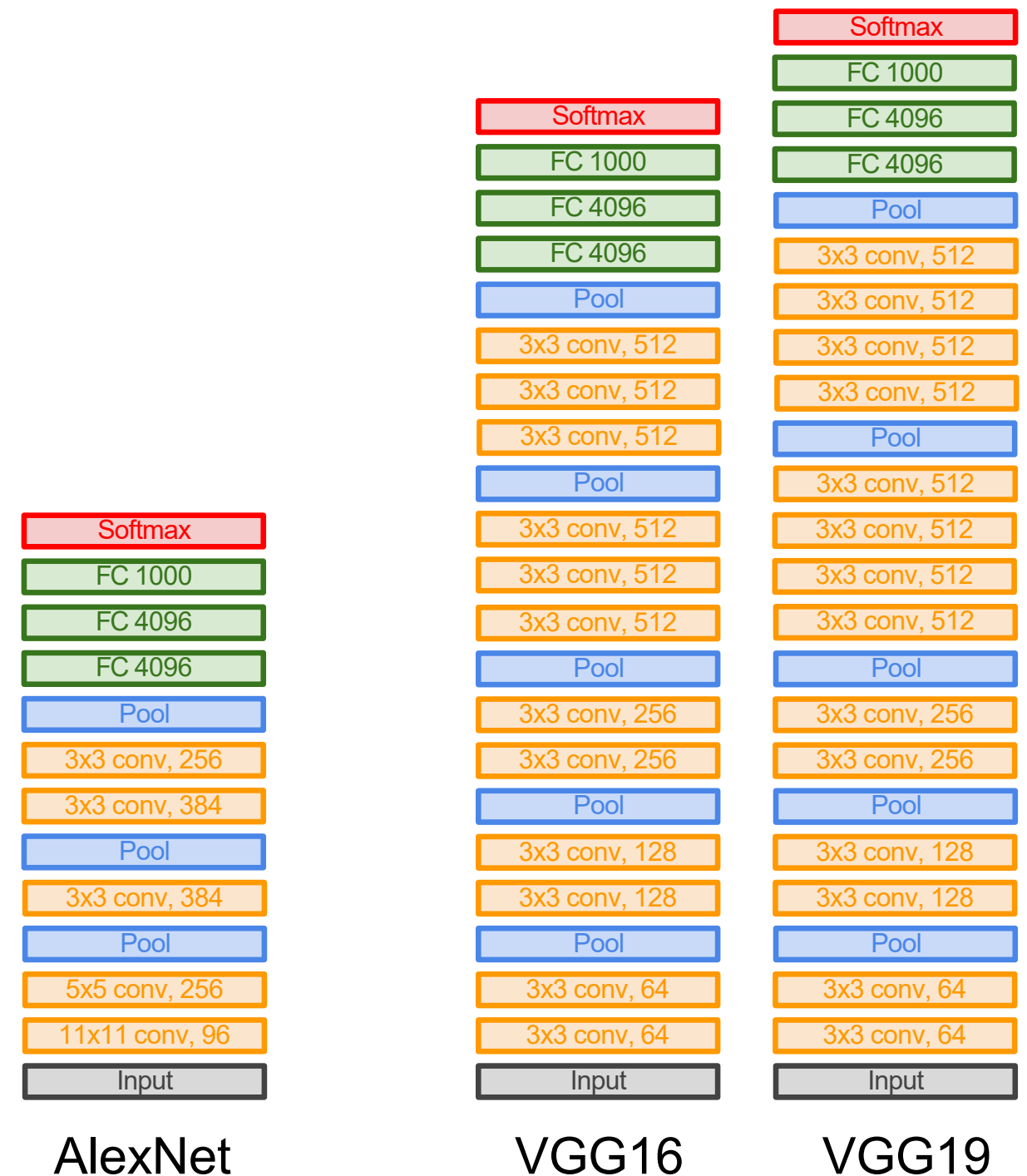
→ 16-19层 (VGG16Net)

仅采用3x3卷积步长1、填充1
及2x2最大池化步长2

ILSVRC'13数据集前5错误率

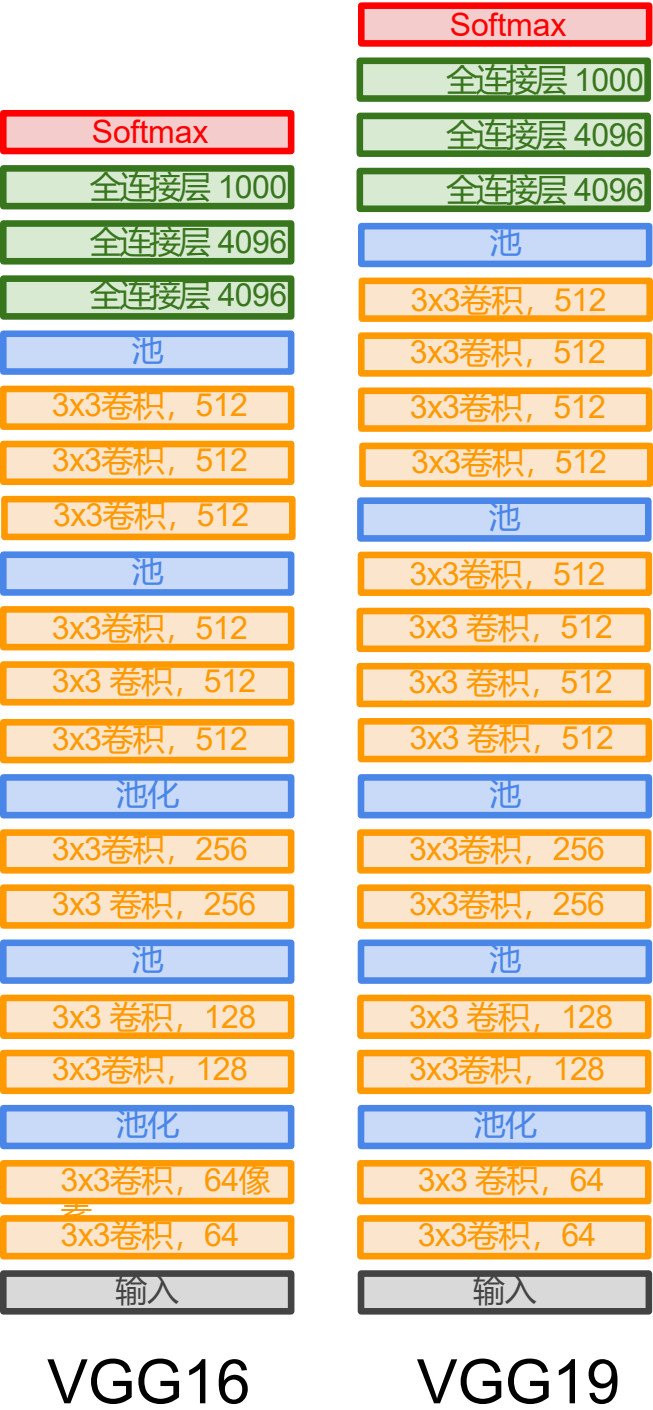
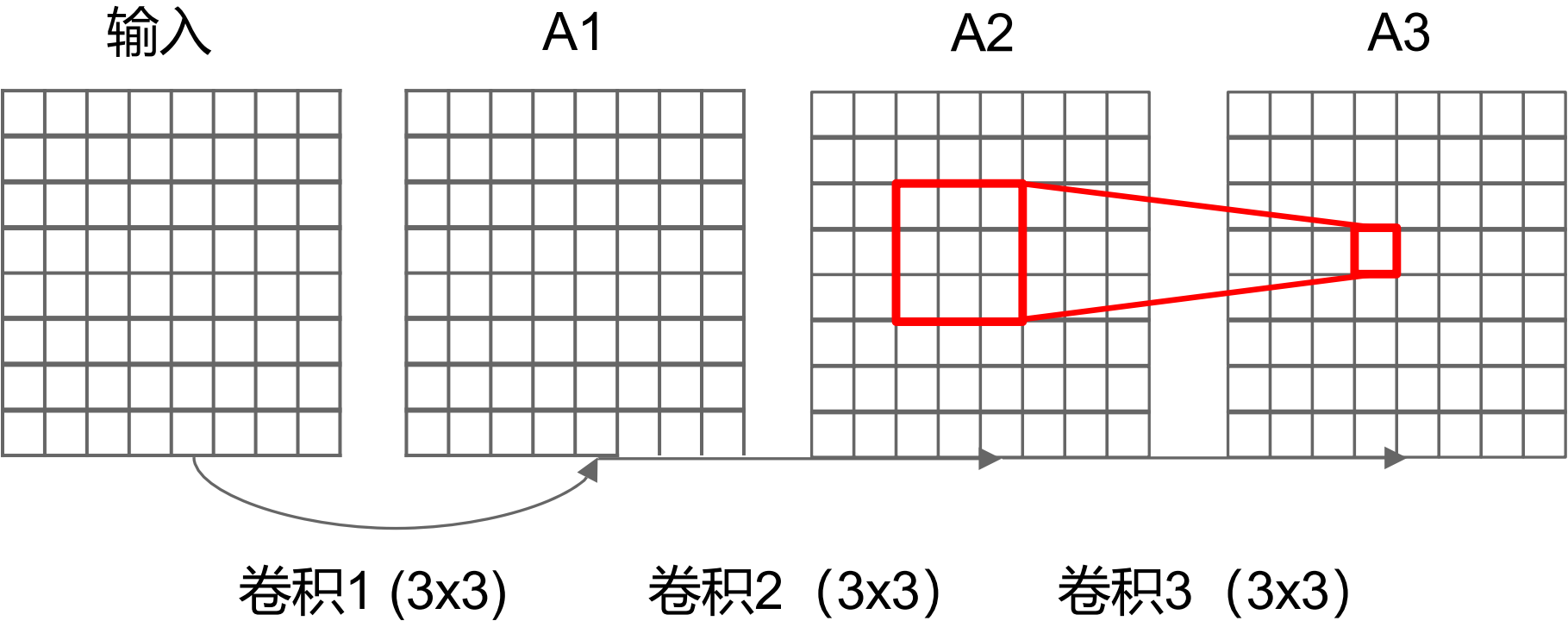
11.7% (ZFNet)

→ ILSVRC'14数据集前5错误率
降至7.3%



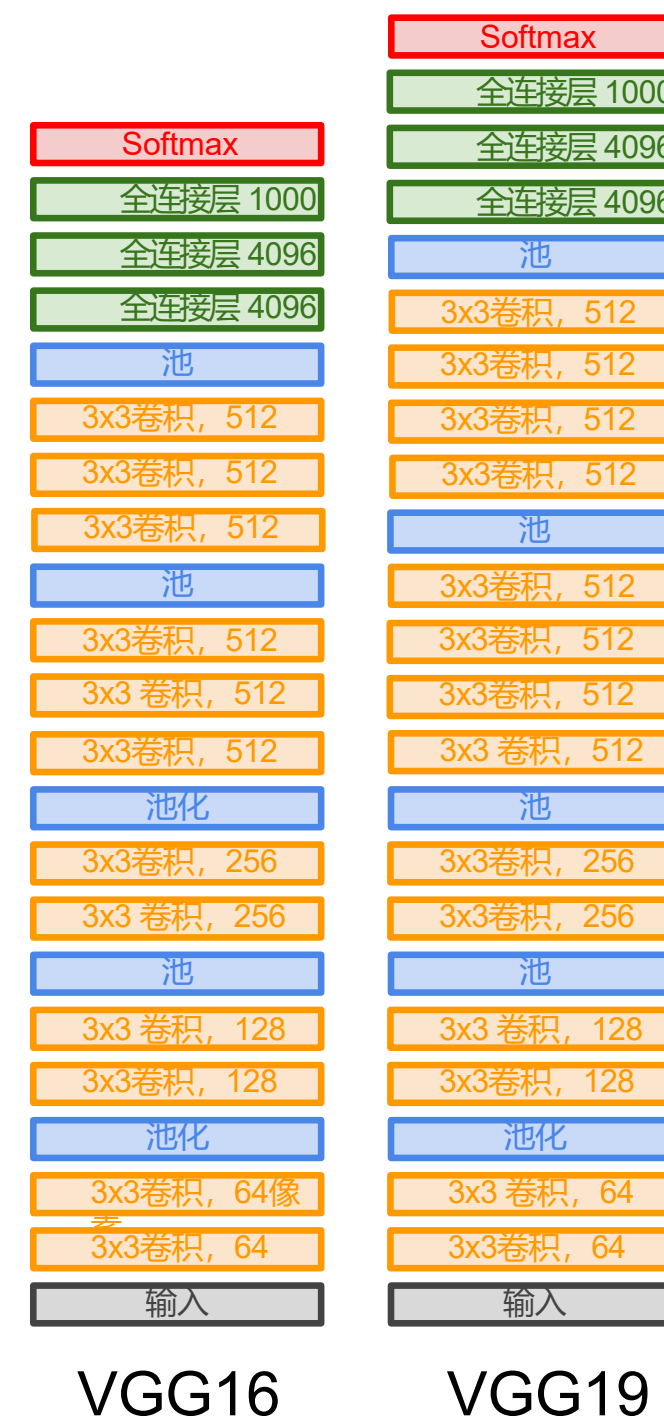
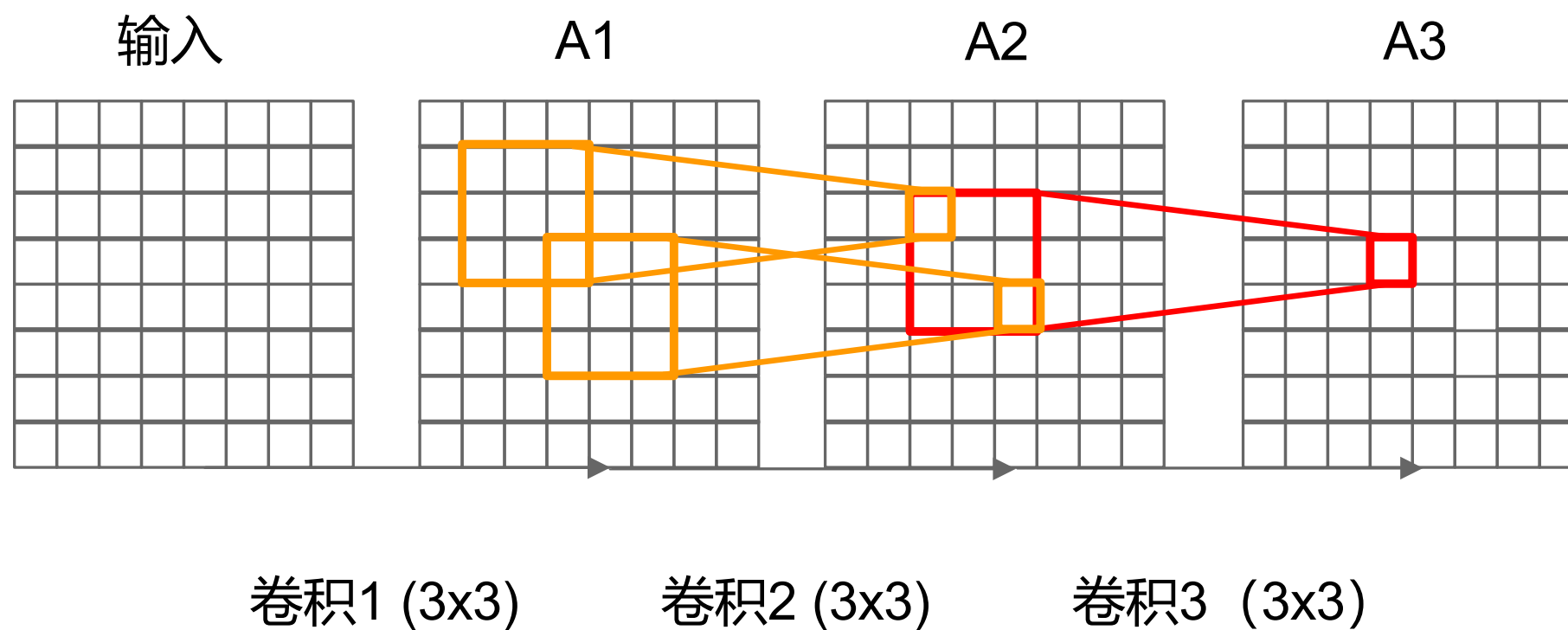
案例研究：VGGNet

问：三个3x3卷积层（步长为1）的有效感受野是多少？



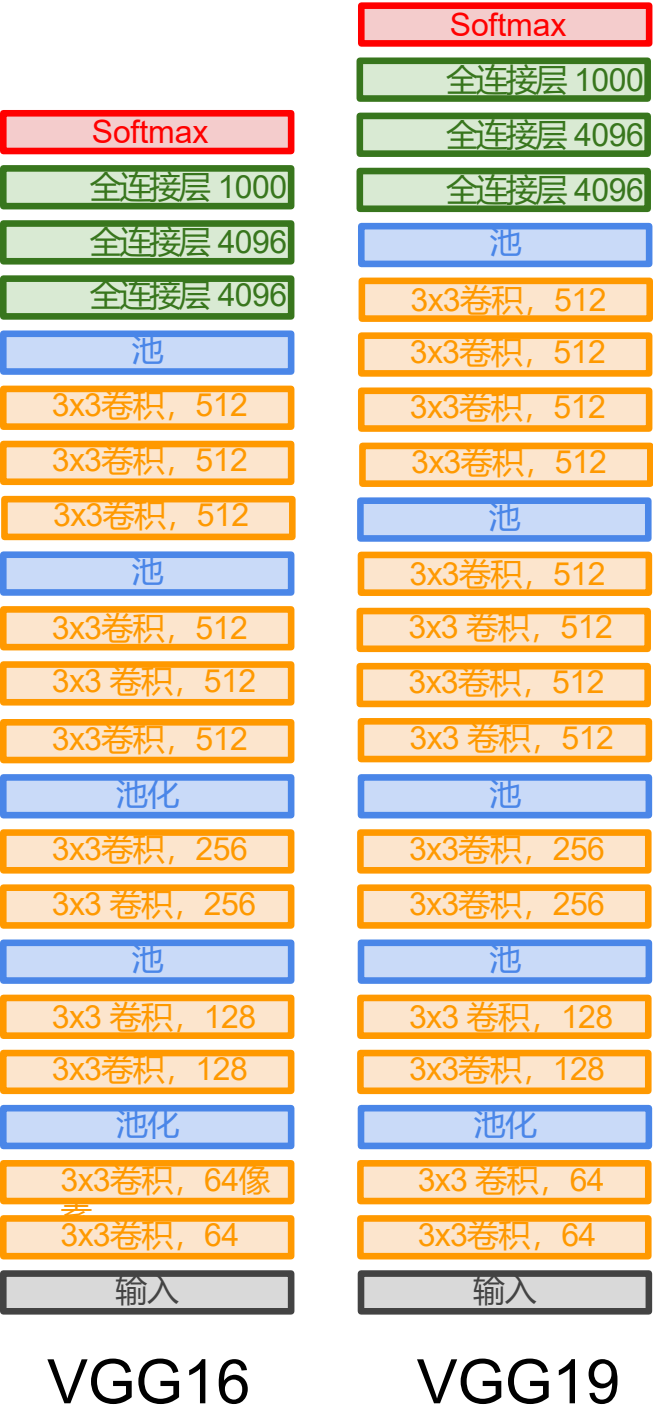
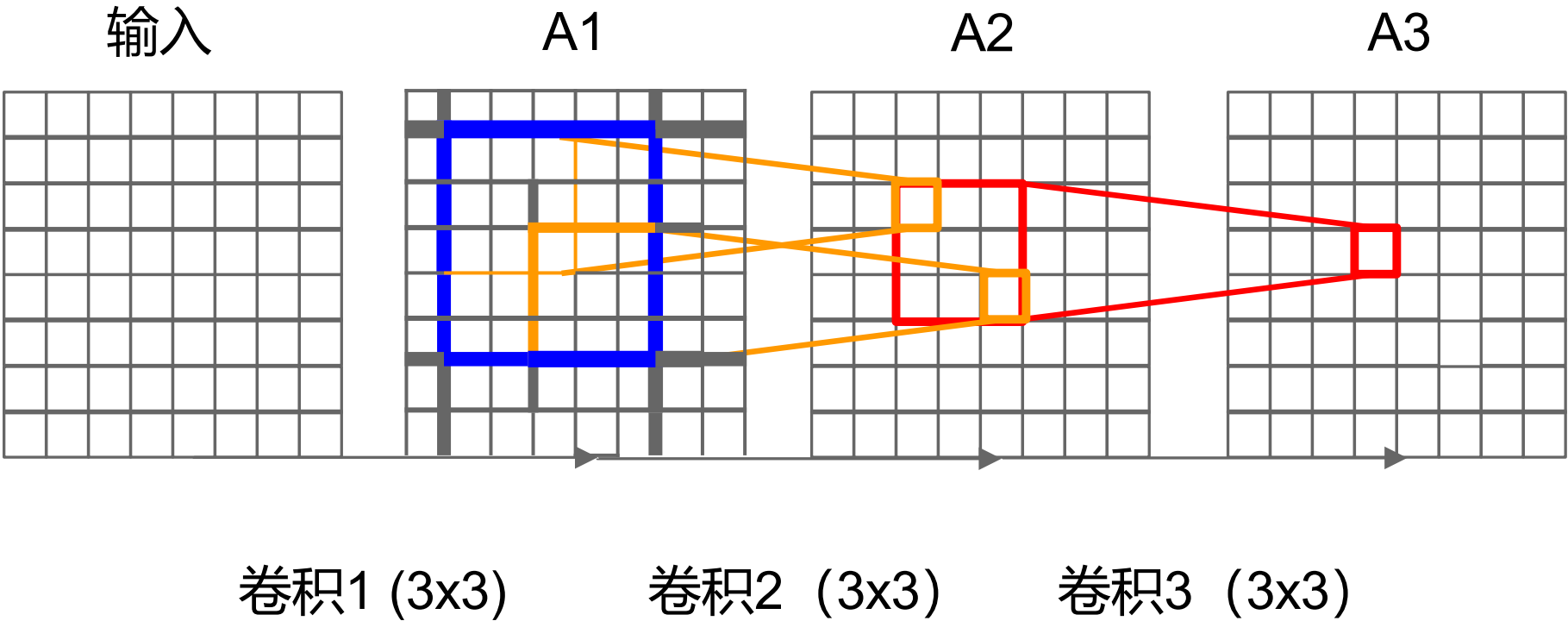
案例研究：VGGNet

问：三个3x3卷积层（步长为1）的有效感受野是多少？



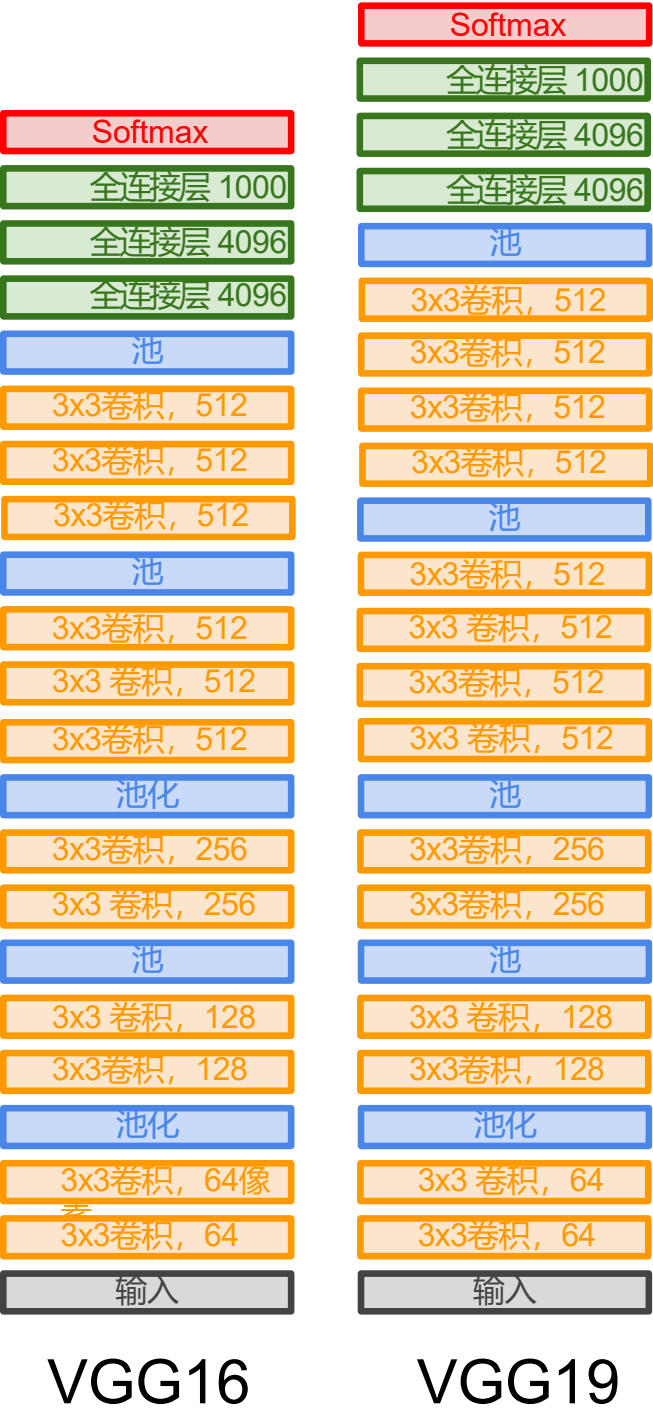
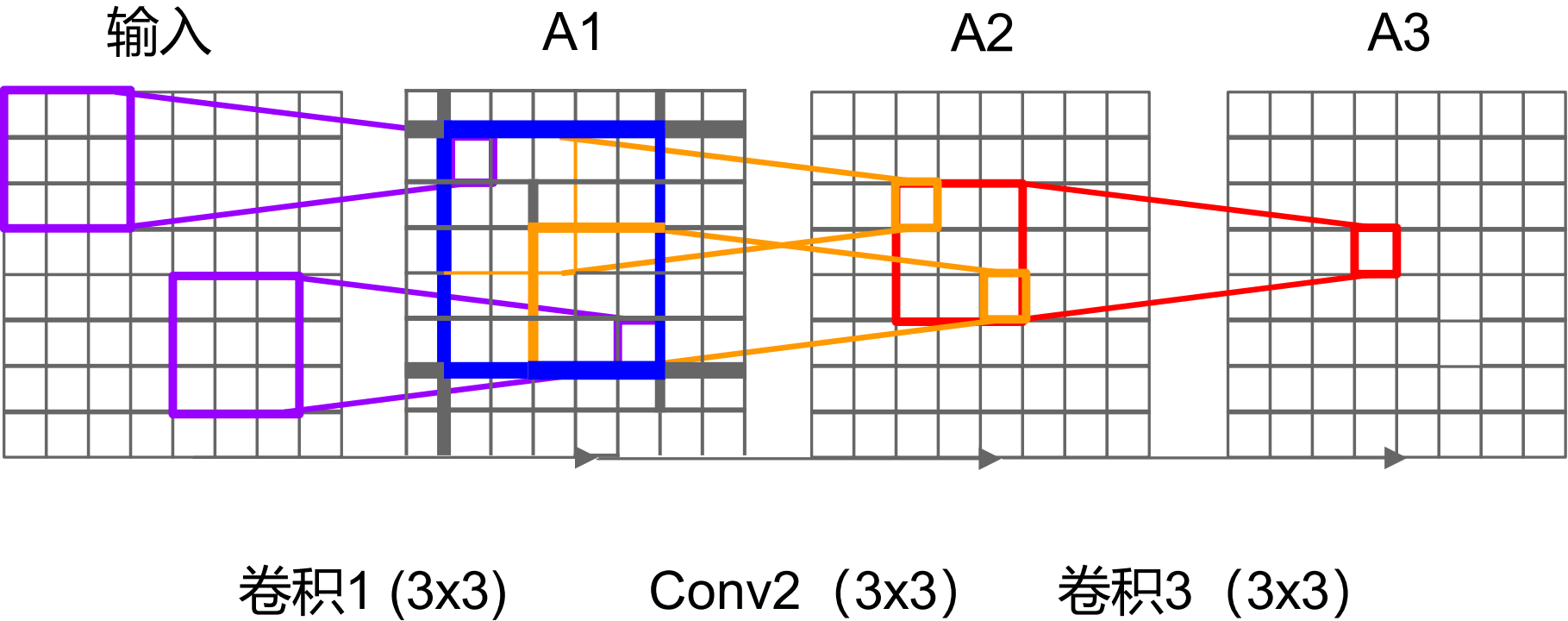
案例研究：VGGNet

问：三个3x3卷积层（步长为1）的有效感受野是多少？



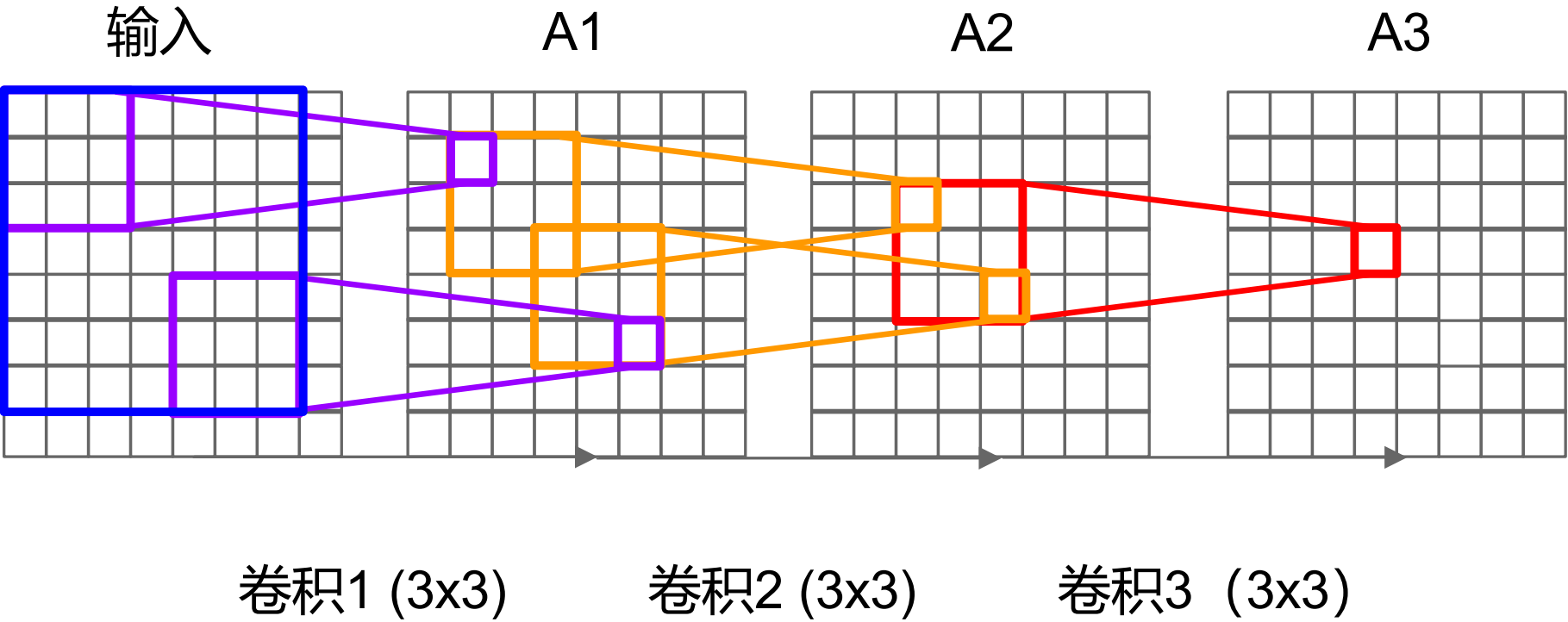
案例研究：VGGNet

问：三个3x3卷积层（步长为1）的有效感受野是多少？



案例研究：VGGNet

问：三个3x3卷积层（步长为1）的有效感受野是多少？



VGG16

VGG19

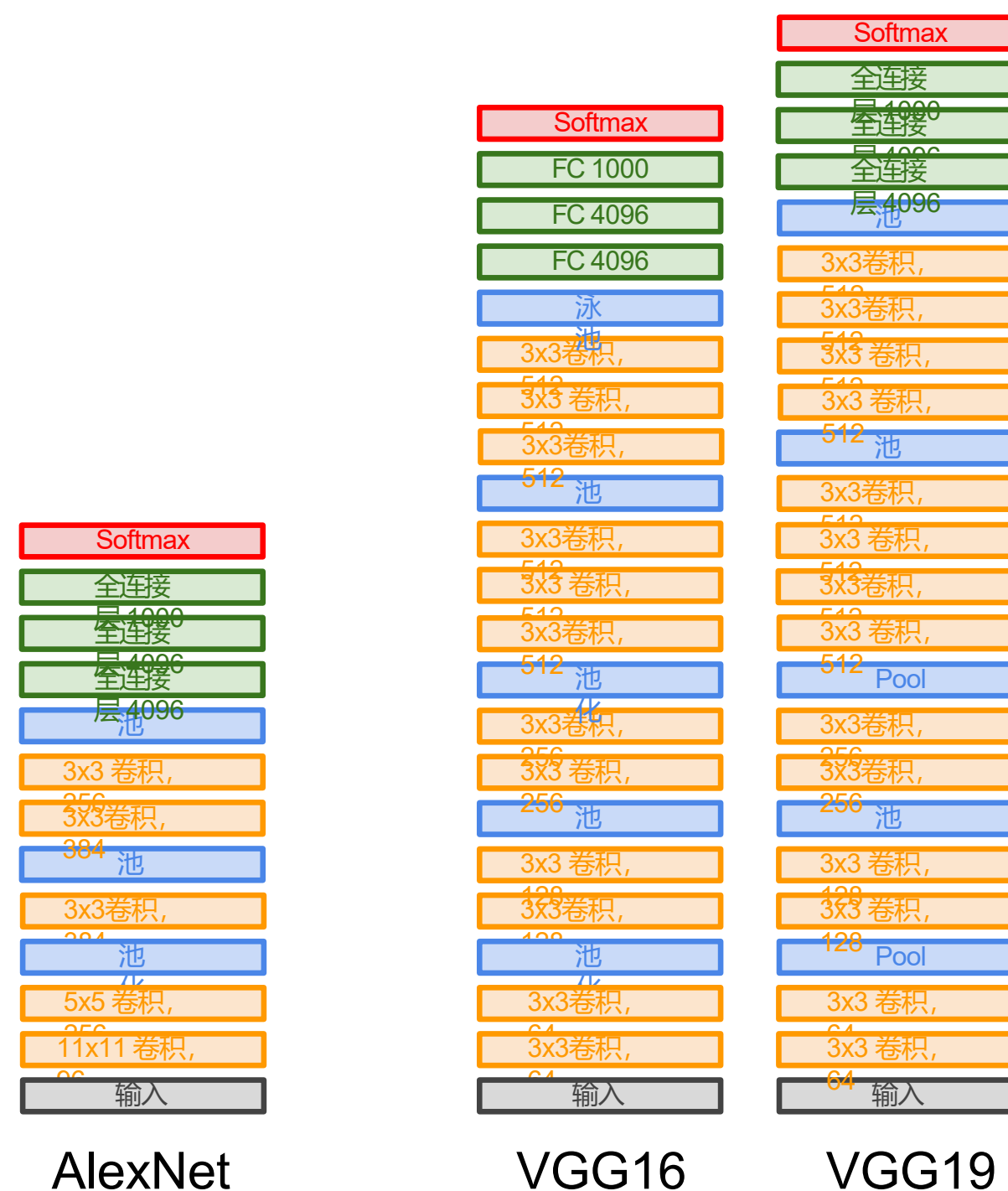
案例研究：VGGNet

问：为何采用较小的卷积核？（3x3卷积）

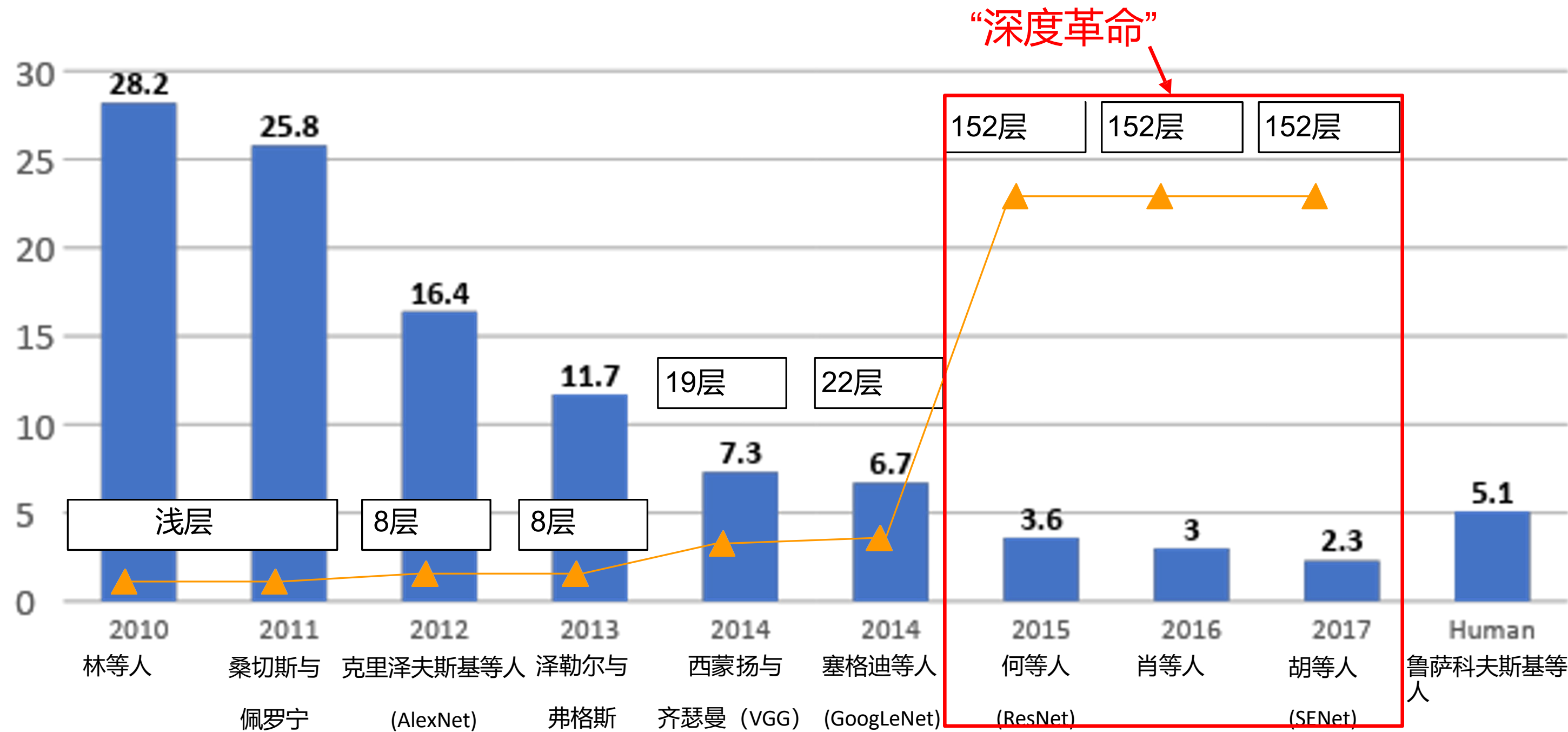
三个3x3卷积层（步长1）的堆叠结构，其有效感受野等同于单个7x7卷积层

但更深层，更多非线性变换

参数更少: $3^3 C^2$ 相比于 $7^2 C^2$



ImageNet大规模视觉识别挑战赛（ILSVRC）获奖者

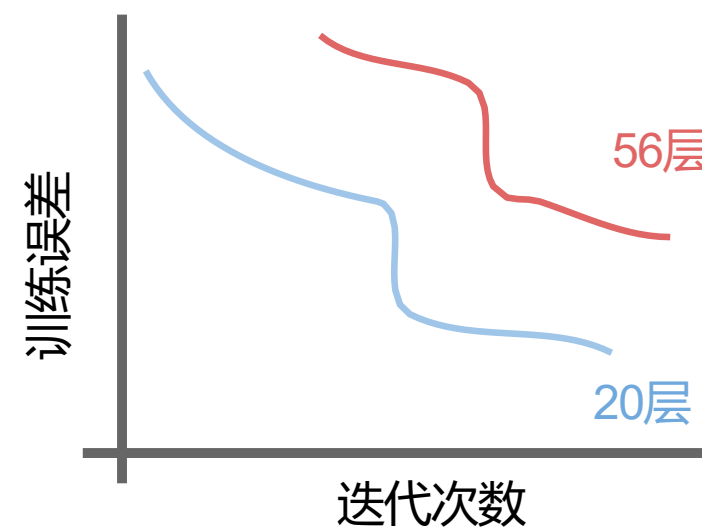
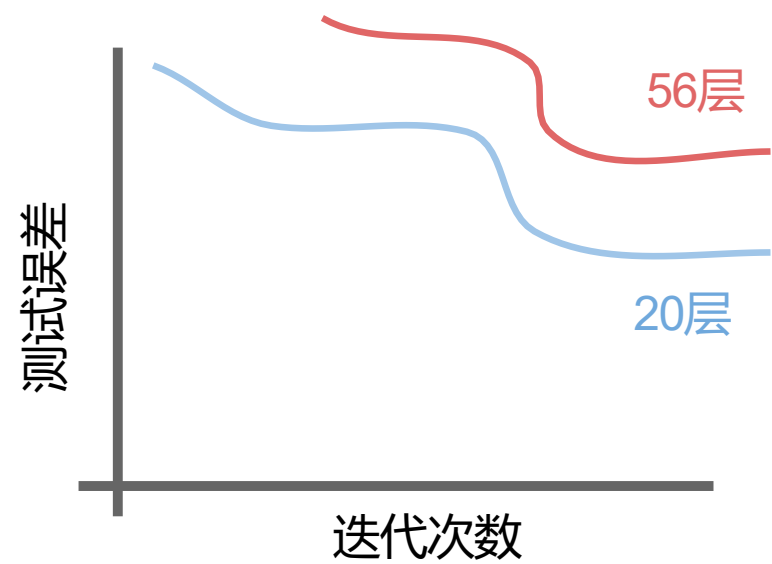


案例研究：ResNet

当我们在"普通"卷积神经网络上继续堆叠更深层时会发生什么？

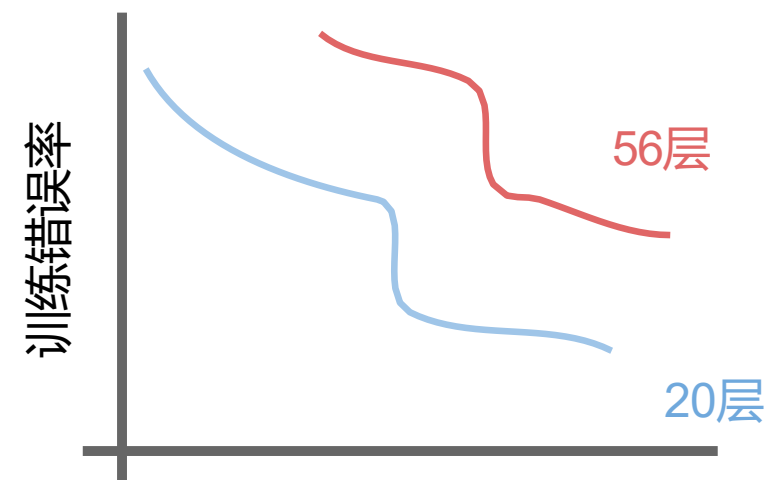
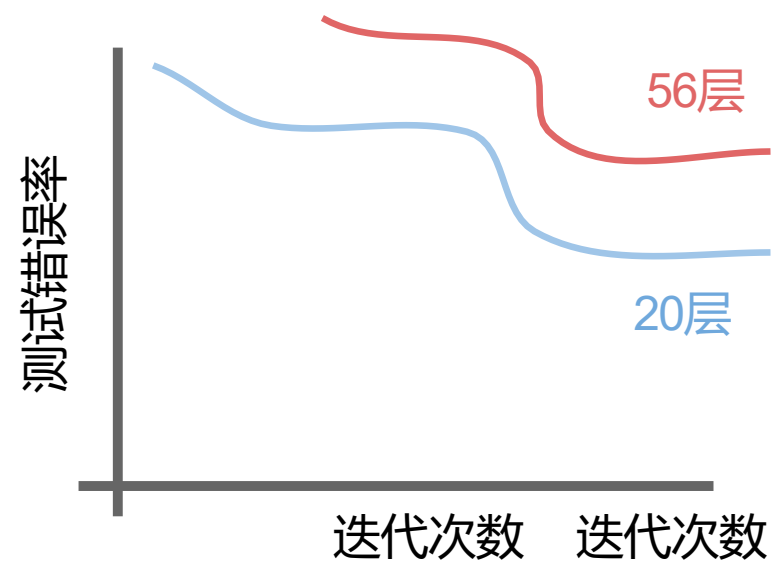
案例研究：ResNet

当我们在"普通"卷积神经网络上继续堆叠更深层时会发生什么？



案例研究：ResNet

当我们在“普通”卷积神经网络上继续堆叠更深层时会发生什么？



56层模型在测试和训练误差上表现更差
-> 更深的模型表现更差，但并非由过拟合引起！

案例研究：ResNet

事实：深度模型比浅层模型具有更强的表征能力（更多参数）。

假设：该问题本质上是*优化*问题，
更深的模型更难优化

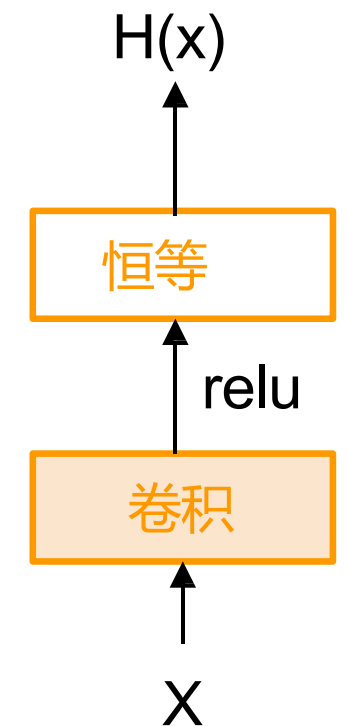
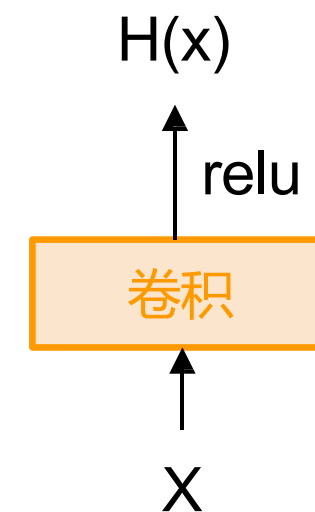
案例研究：ResNet

事实：深度模型比浅层模型具有更强的表征能力（更多参数）。

假设：该问题本质上是优化问题，更深的模型更难优化

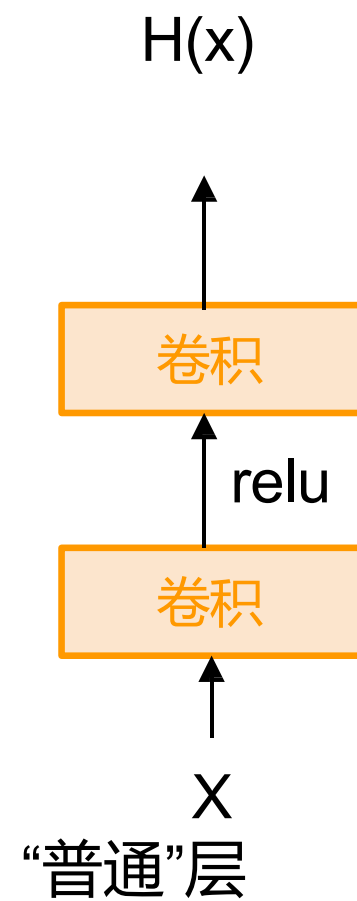
更深的模型需要学习什么才能至少达到浅层模型的性能？

通过构造获得的解决方案是：从较浅的模型中复制已训练好的层，并将额外层设置为恒等映射。



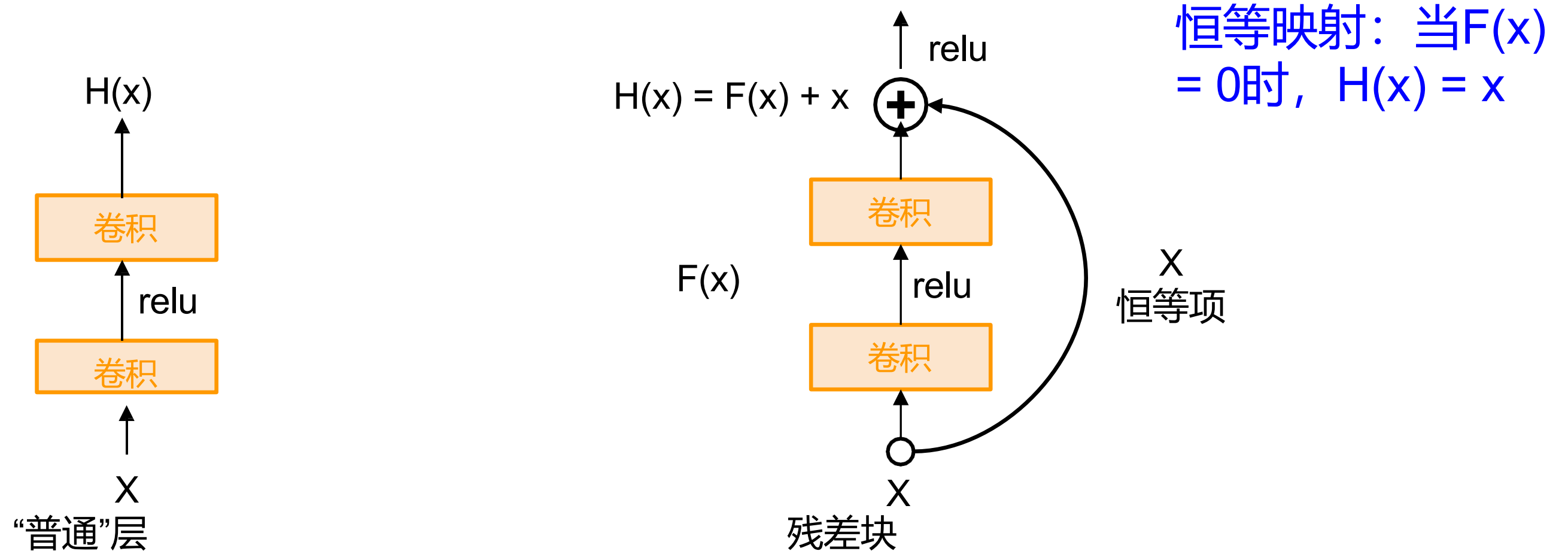
案例研究：ResNet

解决方案：利用网络层拟合残差映射，而非直接尝试拟合期望的底层映射



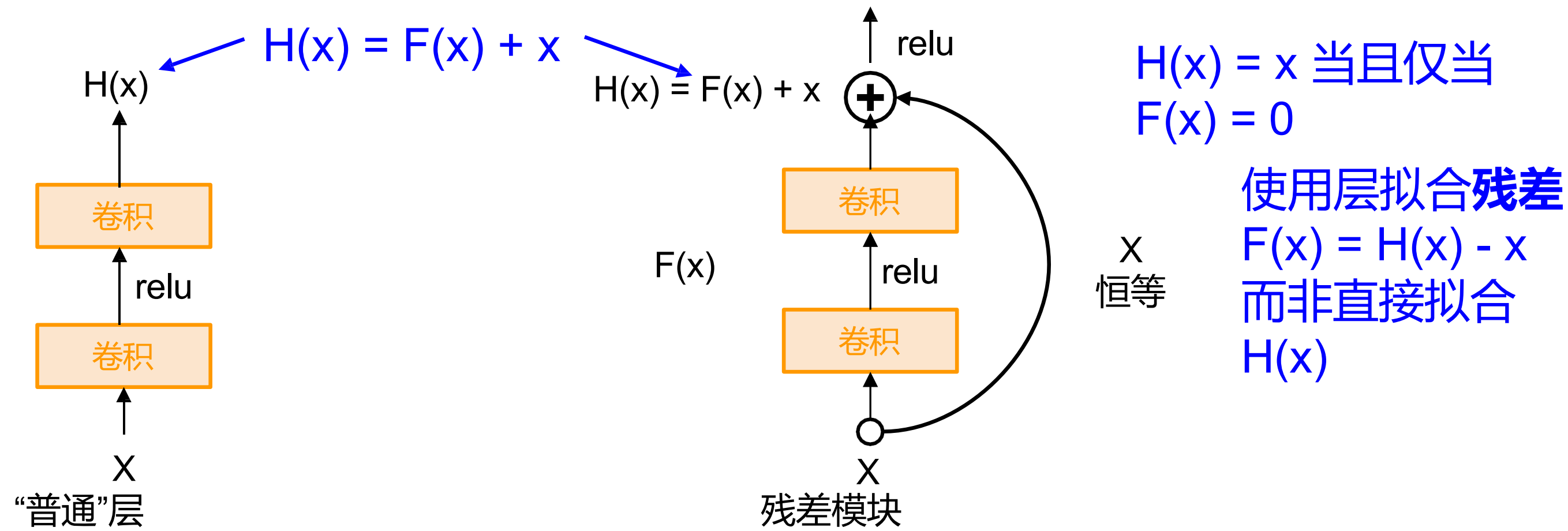
案例研究：ResNet

解决方案：利用网络层拟合残差映射，而非直接尝试拟合期望的底层映射



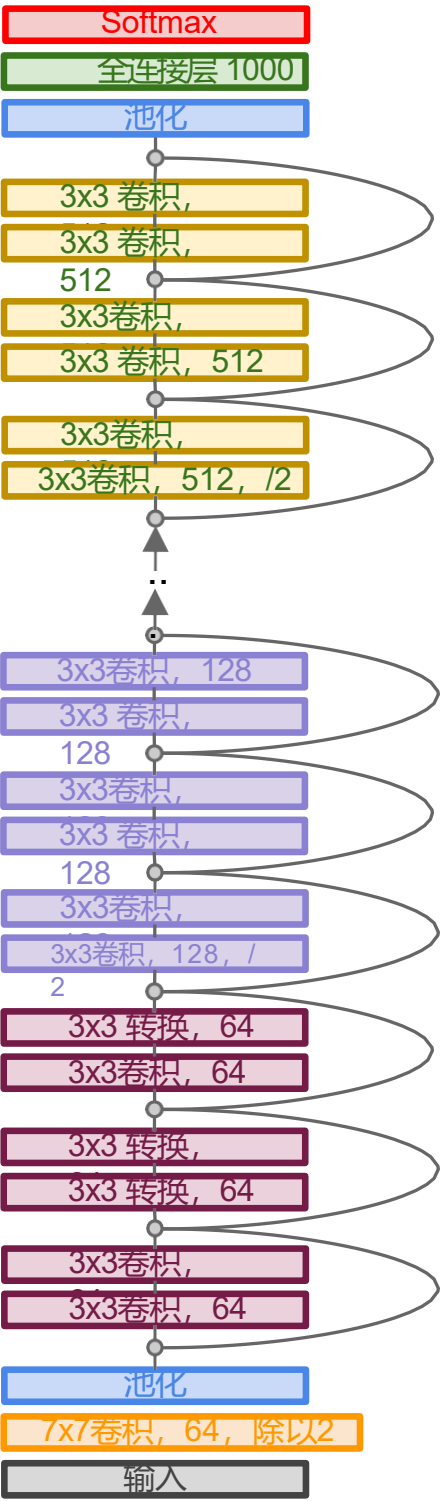
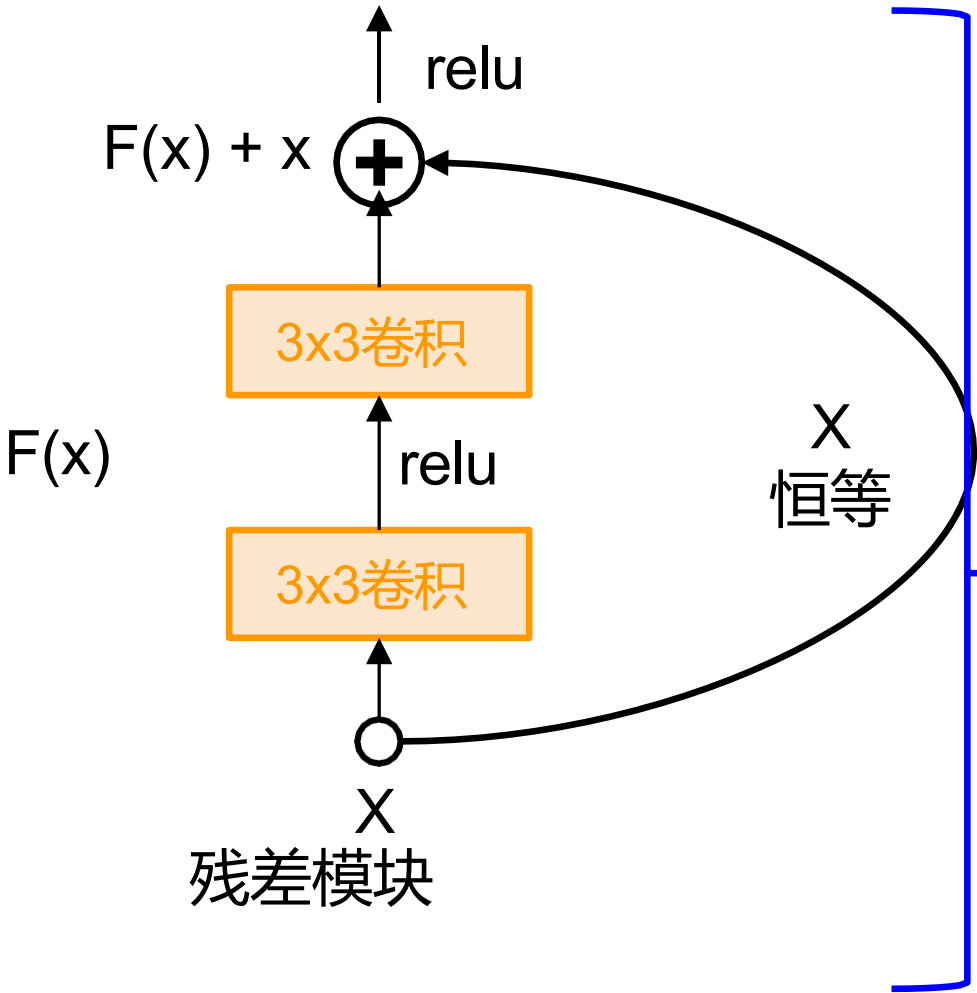
案例研究：ResNet

解决方案：利用网络层拟合残差映射，而非直接尝试拟合期望的底层映射
恒等映射：



案例研究：ResNet

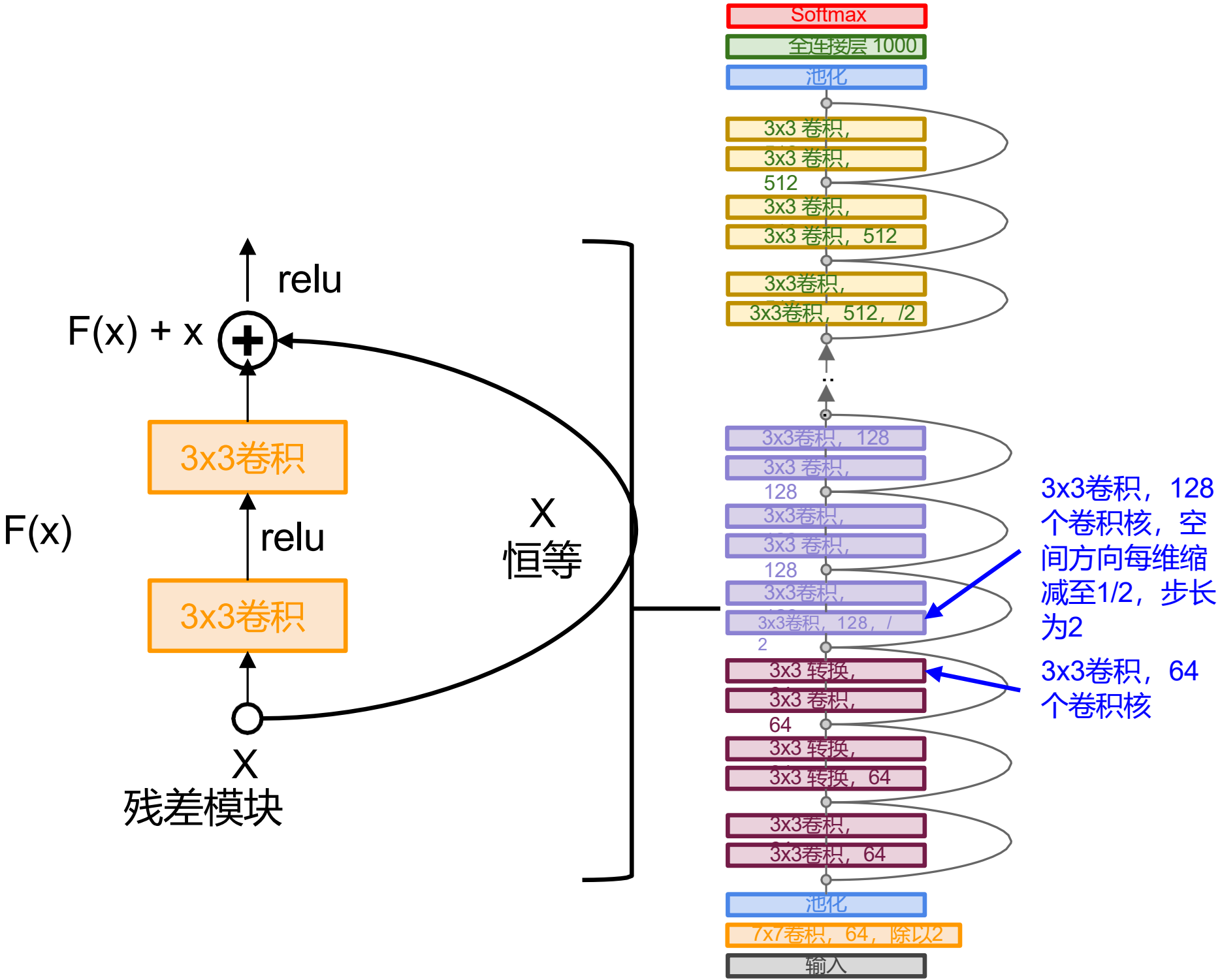
- 完整ResNet架构：
- 堆叠残差块
 - 每个残差块包含两个3x3卷积层



案例研究：ResNet

完整ResNet架构：

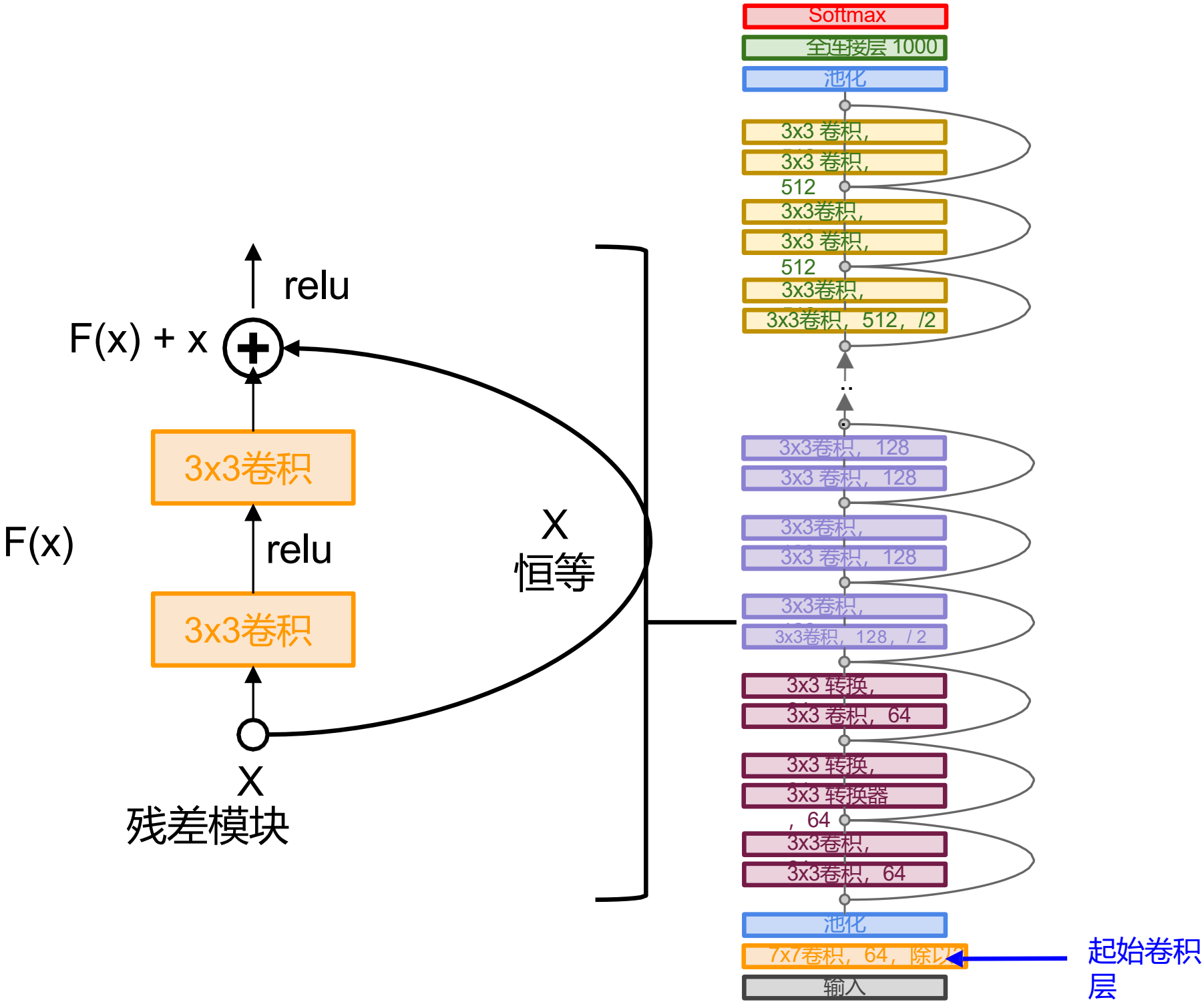
- 堆叠残差块
- 每个残差块包含两个3x3卷积层
- 周期性地将卷积核数量加倍并进行空间下采样
空间方向采用步长2
(各维度减半) 激活体积缩减一半。



案例研究：ResNet

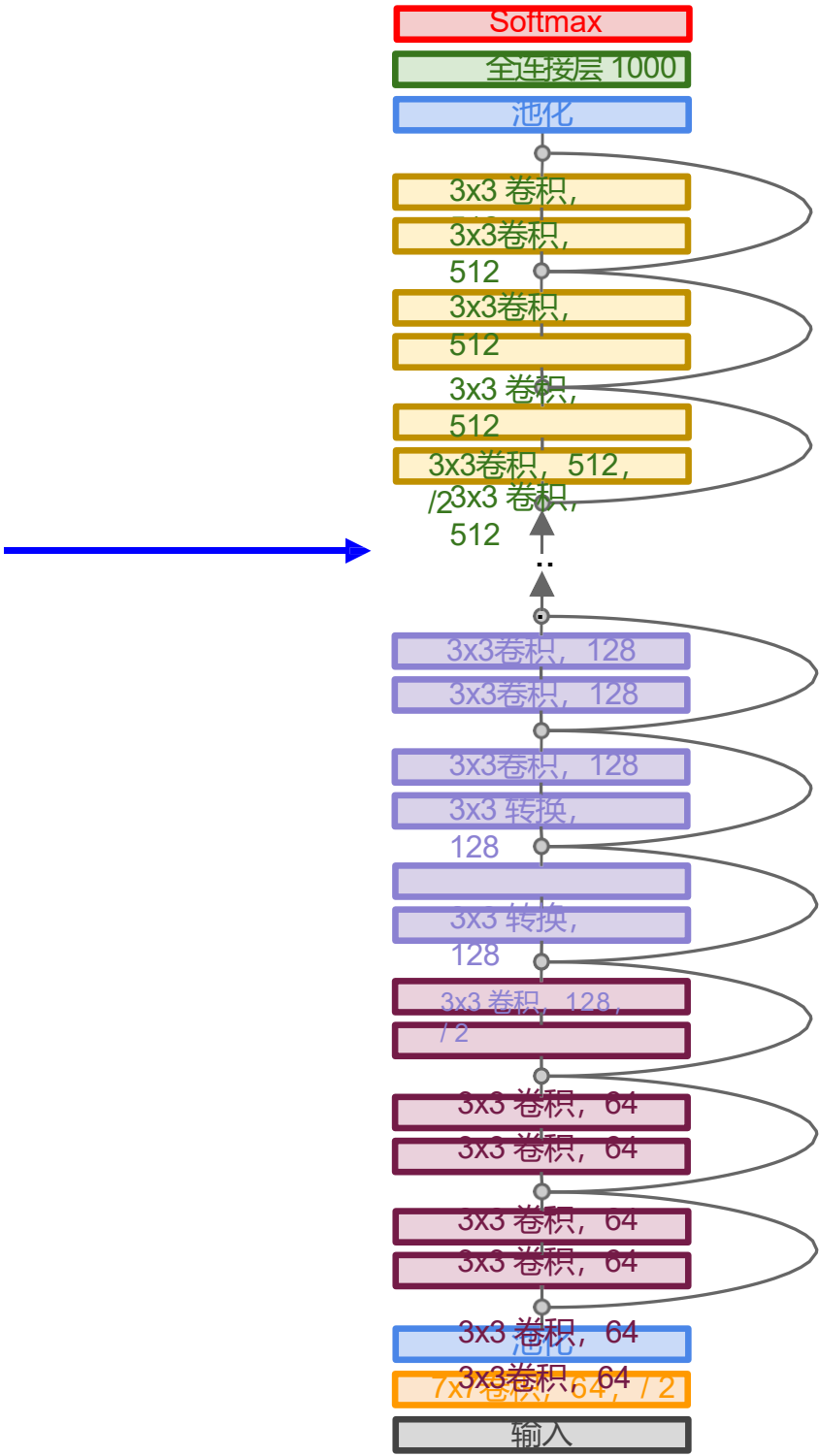
完整ResNet架构：

- 堆叠残差模块
- 每个残差块包含两个3x3卷积层
- 周期性地将卷积核数量加倍并进行空间下采样
空间方向采用步长2
(各维度缩减至原尺寸的1/2)
- 添加额外卷积层



案例研究： ResNet

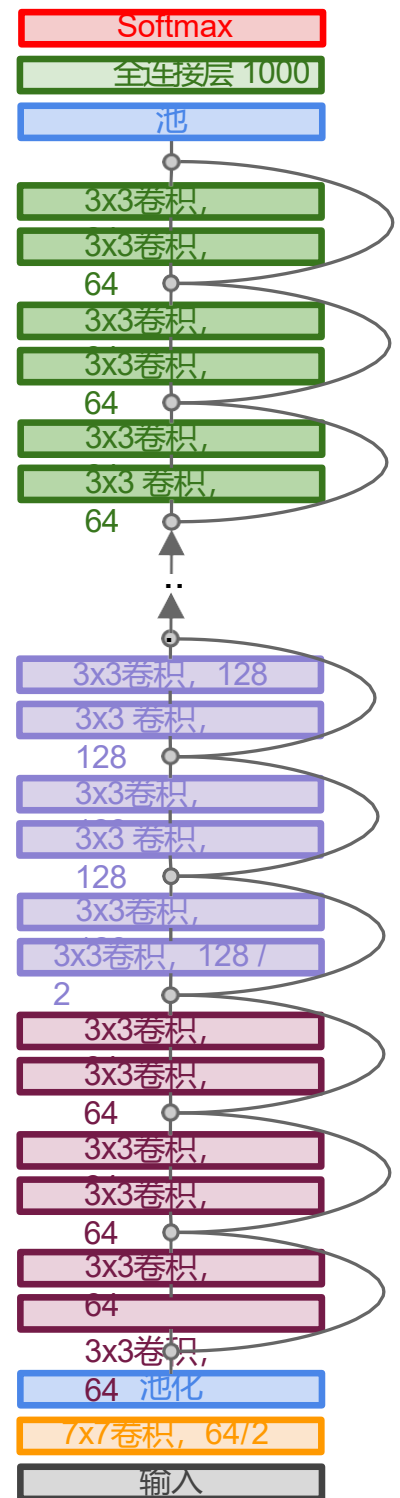
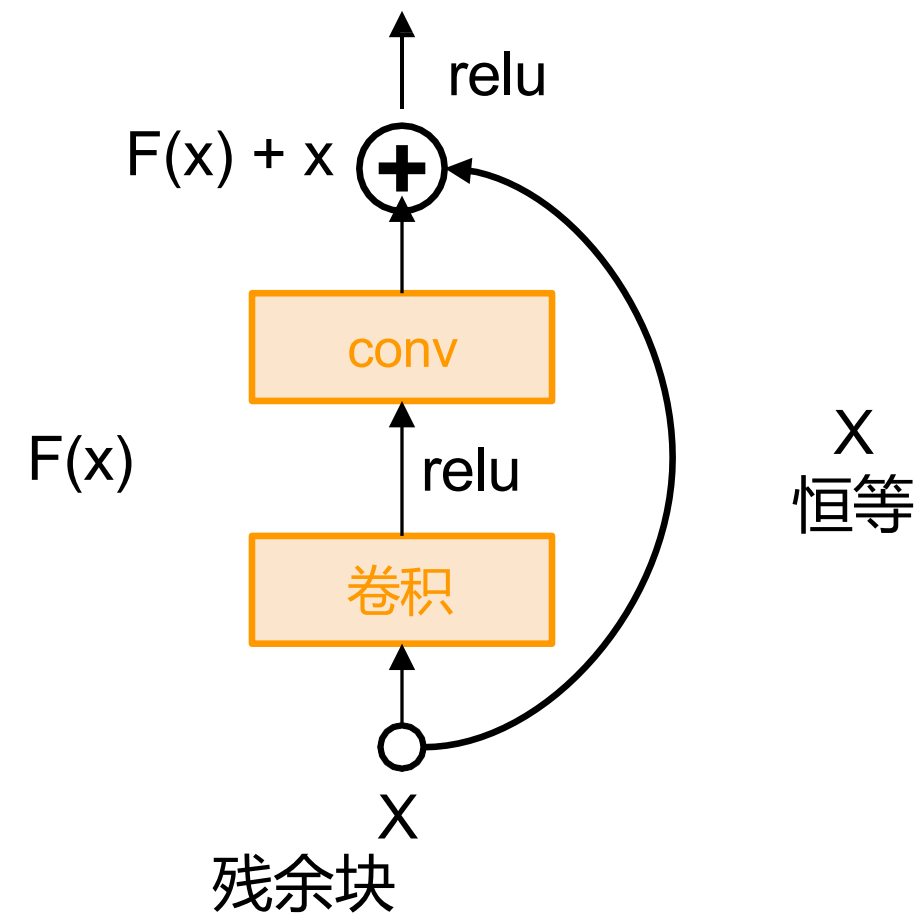
总层数为18、 34、 50、
101或152层



案例研究：ResNet

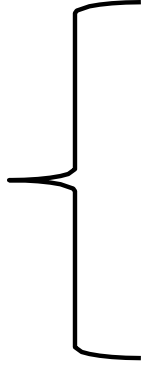
基于残差连接的超深度神经网络

- 用于ImageNet的152层模型
- ILSVRC'15分类竞赛冠军（前五类错误率3.57%）
- 横扫ILSVRC'15与COCO'15所有分类与检测竞赛！



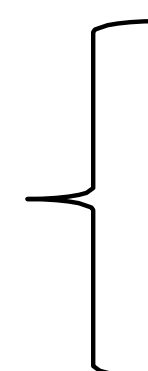
两大主题

如何构建卷积神经网络?



卷积神经网络中的层结构
激活函数
卷积神经网络架构
权重初始化

如何训练卷积神经网络?



数据预处理
数据增强
迁移学习
超参数选择

权重初始化案例：数值过小

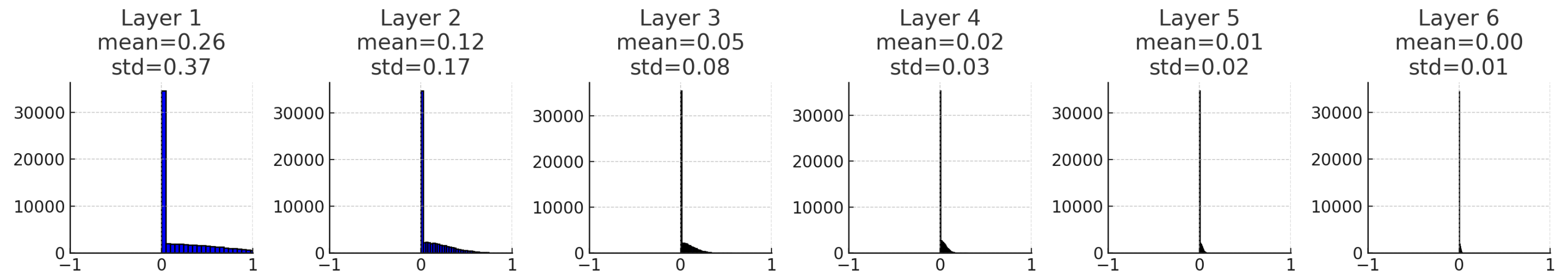
```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout) # Small weight init
    x = np.maximum(0, x.dot(W)) # ReLU activation
    hs.append(x)
```

6层神经网络（隐藏层尺寸为4096）的前向传播

权重初始化案例：数值过小

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.01 * np.random.randn(Din, Dout) # Small weight init
    x = np.maximum(0, x.dot(W)) # ReLU activation
    hs.append(x)
```

更深层网络中的所有激活值趋近于零



权重初始化案例：数值过大

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout) # Small weight init
    x = np.maximum(0, x.dot(W)) # ReLU activation
    hs.append(x)
```

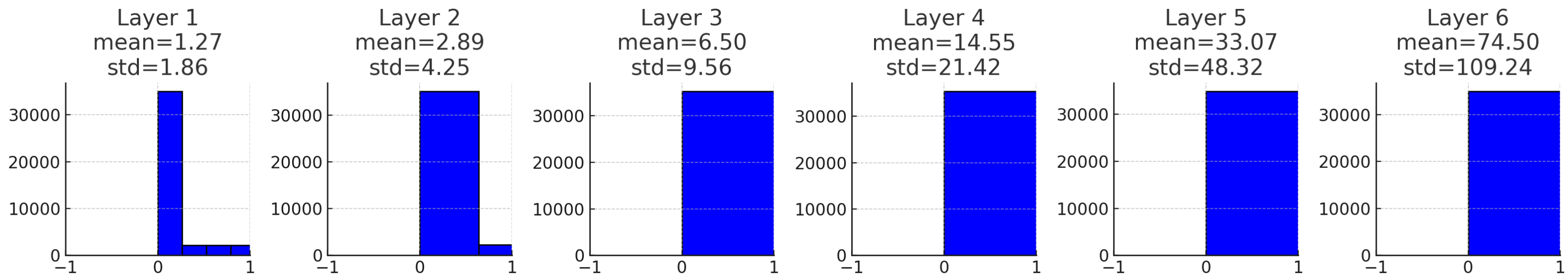
将初始权重的标准差从
0.01增加至0.05

权重初始化案例：数值过大

```
dims = [4096] * 7
hs = []
x = np.random.randn(16, dims[0])
# Forward pass with ReLU activation
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = 0.05 * np.random.randn(Din, Dout) # Small weight init
    x = np.maximum(0, x.dot(W)) # ReLU activation
    hs.append(x)
```

激活函数快速发散

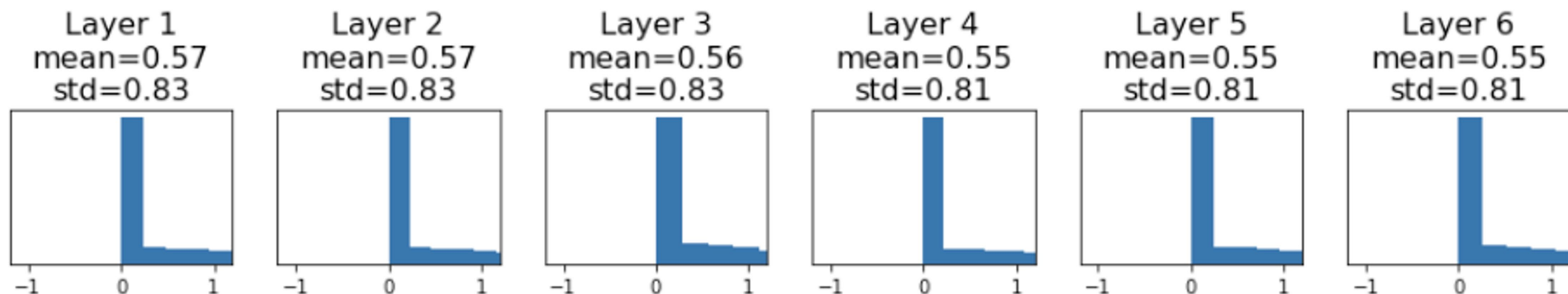
将初始权重的标准差从
0.01提高至0.05



一种解决方案：Kaiming/MSRA初始化

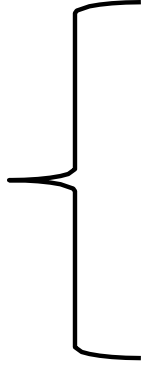
```
dims = [4096] * 7  ReLU修正: std = sqrt(2 / Din)
hs = []
x = np.random.randn(16, dims[0])
for Din, Dout in zip(dims[:-1], dims[1:]):
    W = np.random.randn(Din, Dout) * np.sqrt(2/Din)
    x = np.maximum(0, x.dot(W))
    hs.append(x)
```

"恰到好处": 所有层的激活函数都经过了精妙缩放!



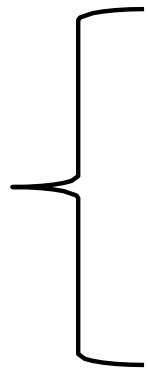
两大主题

如何构建卷积神经网络?



卷积神经网络中的层结构
激活函数
卷积神经网络架构
权重初始化

如何训练卷积神经网络?



数据预处理
数据增强
迁移学习
超参数选择

图像归一化简述：对每个通道进行中心化和缩放

- 减去各通道均值
除以各通道标准差
- 需预先计算数据集中每个像素通道的均值与标准差

```
norm_pixel[i,j,c] = (pixel[i,j,c] - np.mean(pixel[:, :, c])) / np.std(pixel[:, :, c])
```


两大主题

如何构建卷积神经网络?

卷积神经网络中的层结构
激活函数
卷积神经网络架构
权重初始化

如何训练卷积神经网络?

数据预处理
数据增强
迁移学习
超参数选择

数据增强：正则化

训练：引入某种随机性

$$y = f_W(x, z)$$

其中 x 是输入， z 是随机变量（如 Dropout mask、噪声等）。这样做的目的：让模型在不同的随机条件下都能表现良好，提高泛化能力。

测试：消除随机性（有时近似处理）

$$y = f(x) = E_z[f(x, z)] = \int p(z) f(x, z) dz$$

用所有可能的随机情况的加权平均，来得到更稳定的预测。

数据增强：正则化

训练：引入某种随机性

$$y = f_W(x, z)$$

测试：消除随机性（有时近似处理）

$$y = f(x) = E_z[f(x, z)] = \int p(z)f(x, z)dz$$

示例：Dropout

训练：随机丢弃激活值

测试：使用所有激活值并按概率p取平均

数据增强

水平翻转



数据增强

随机裁剪与缩放

训练： 随机裁剪/缩放样本

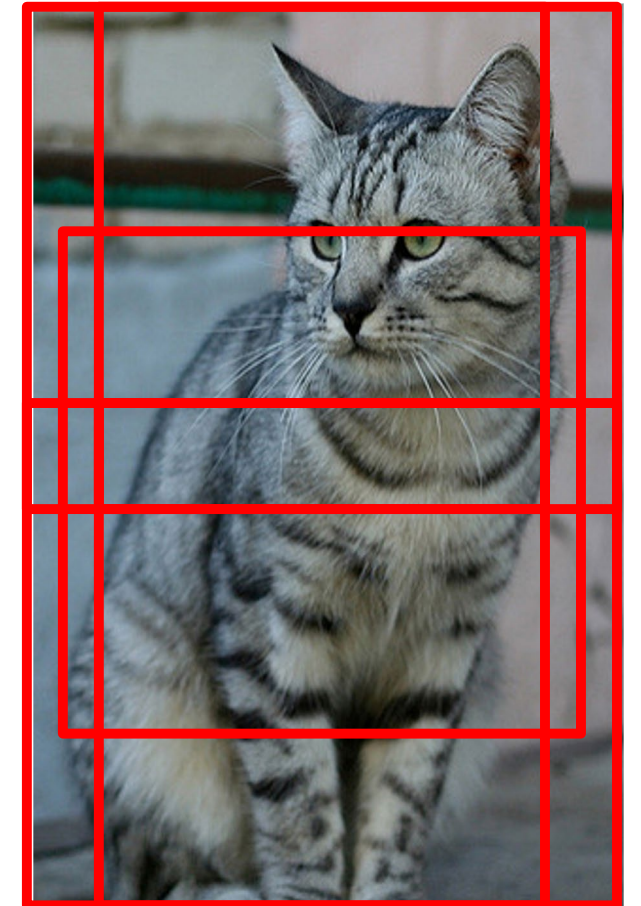
ResNet:

1. 在[256, 480]范围内随机选取L值
2. 调整训练图像尺寸，短边=L
3. 采样随机224×224像素区域

测试时增强： 对固定裁剪集取平均值

ResNet:

1. 将图像按5个比例缩放： {224, 256, 384, 480, 640}
2. 每种尺寸使用10张224×224裁剪图： 4个角点+中心点，并进行镜像翻转



数据增强

颜色抖动

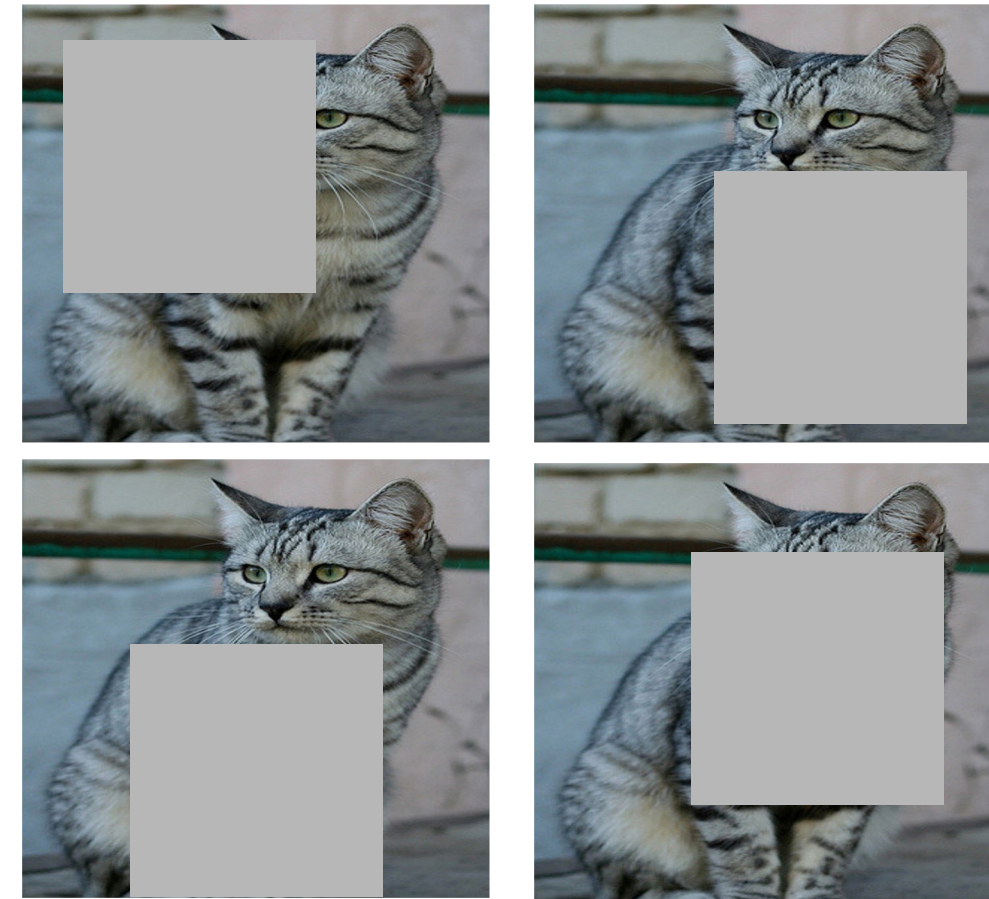
简单：随机化对比度和亮度



数据增强：裁剪

训练：将随机图像区域设为零

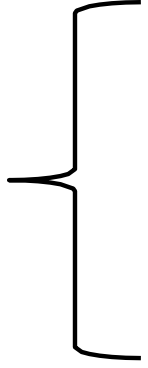
测试：使用完整图像



对CIFAR等小型数据集效果显著，在ImageNet等大型数据集中应用较少

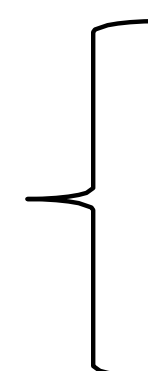
两大主题

如何构建卷积神经网络?



- 卷积神经网络中的层结构
- 激活函数
- 卷积神经网络架构
- 权重初始化

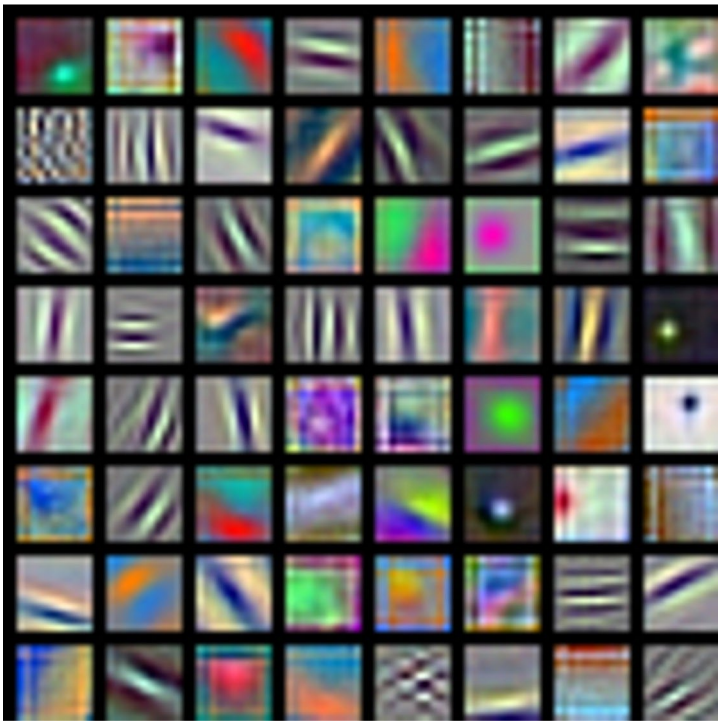
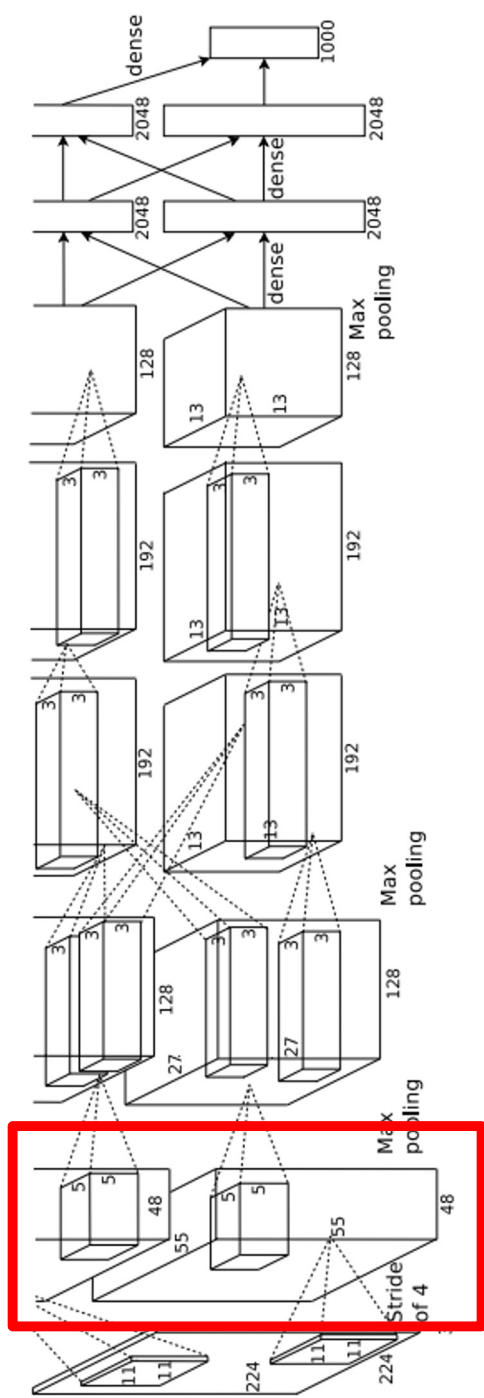
如何训练卷积神经网络?



- 数据预处理
- 数据增强
- 迁移学习
- 超参数选择

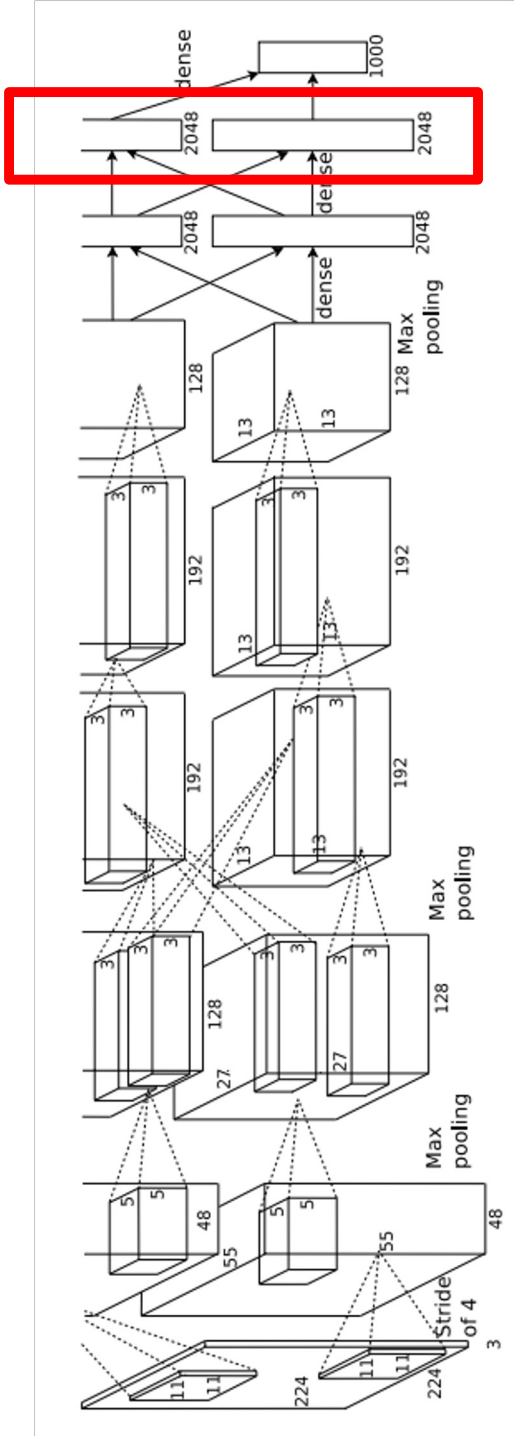
如果数据量不足，该怎么办？
还能训练卷积神经网络吗？

基于卷积神经网络的迁移学习



AlexNet:
64 x 3 x 11 x 11

基于卷积神经网络的迁移学习



测试图像

L2 特征空间中的最近邻算法



卷积神经网络的迁移学习

1. 在ImageNet（或互联网规模数据集）上训练



基于卷积神经网络的迁移学习

1. 在Imagenet上训练



2. 小型数据集 (C类)



重新初始化
并训练

从预训练模型中冻
结这些特征

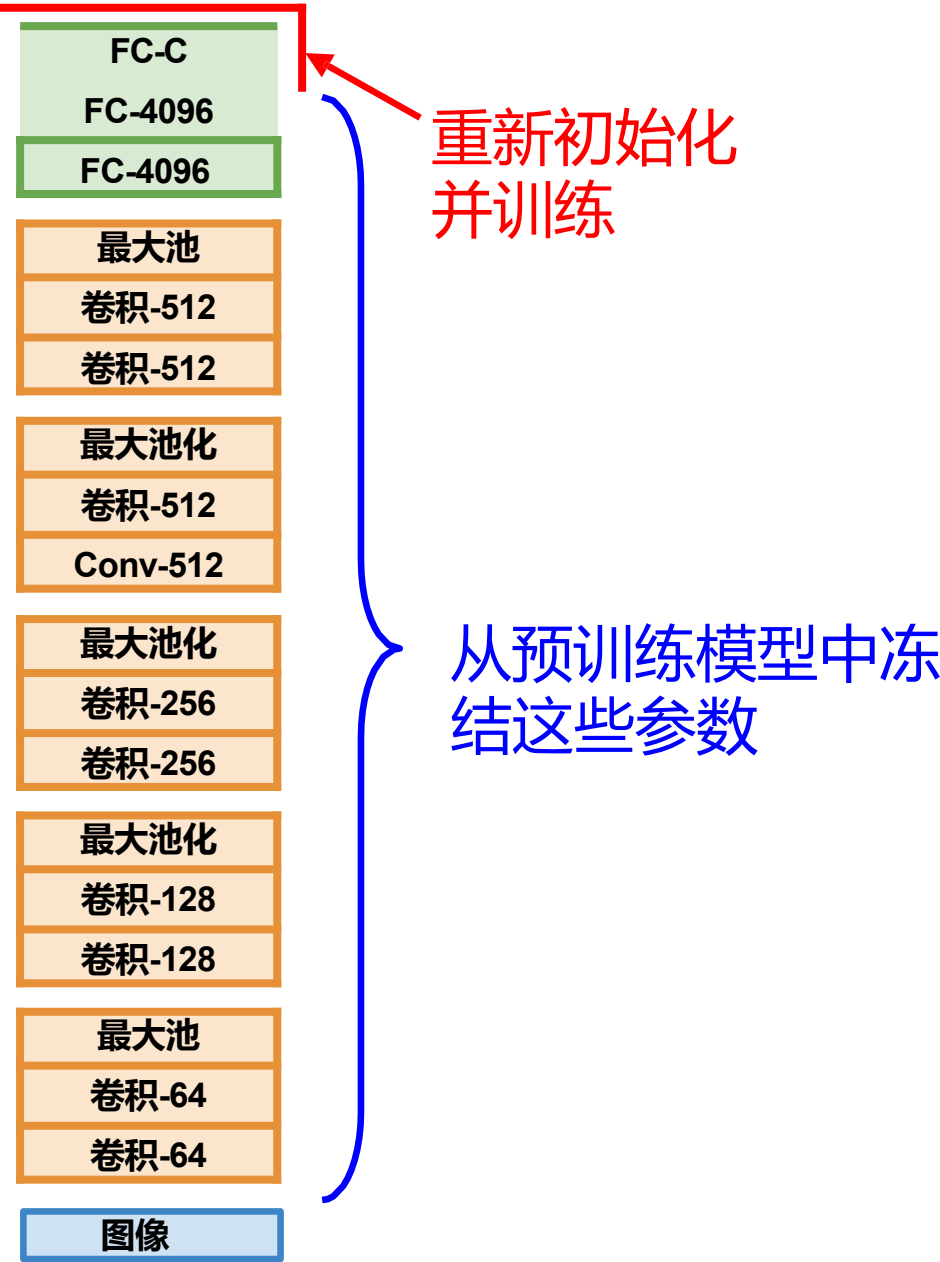
基于卷积神经网络的迁移学习

Donahue等人, 《DeCAF: 通用视觉识别中的深度卷积
激活特征》, ICML 2014Razavian等人, 《现成的卷积
神经网络特征: 识别任务的惊人基线》, CVPR研讨会
2014

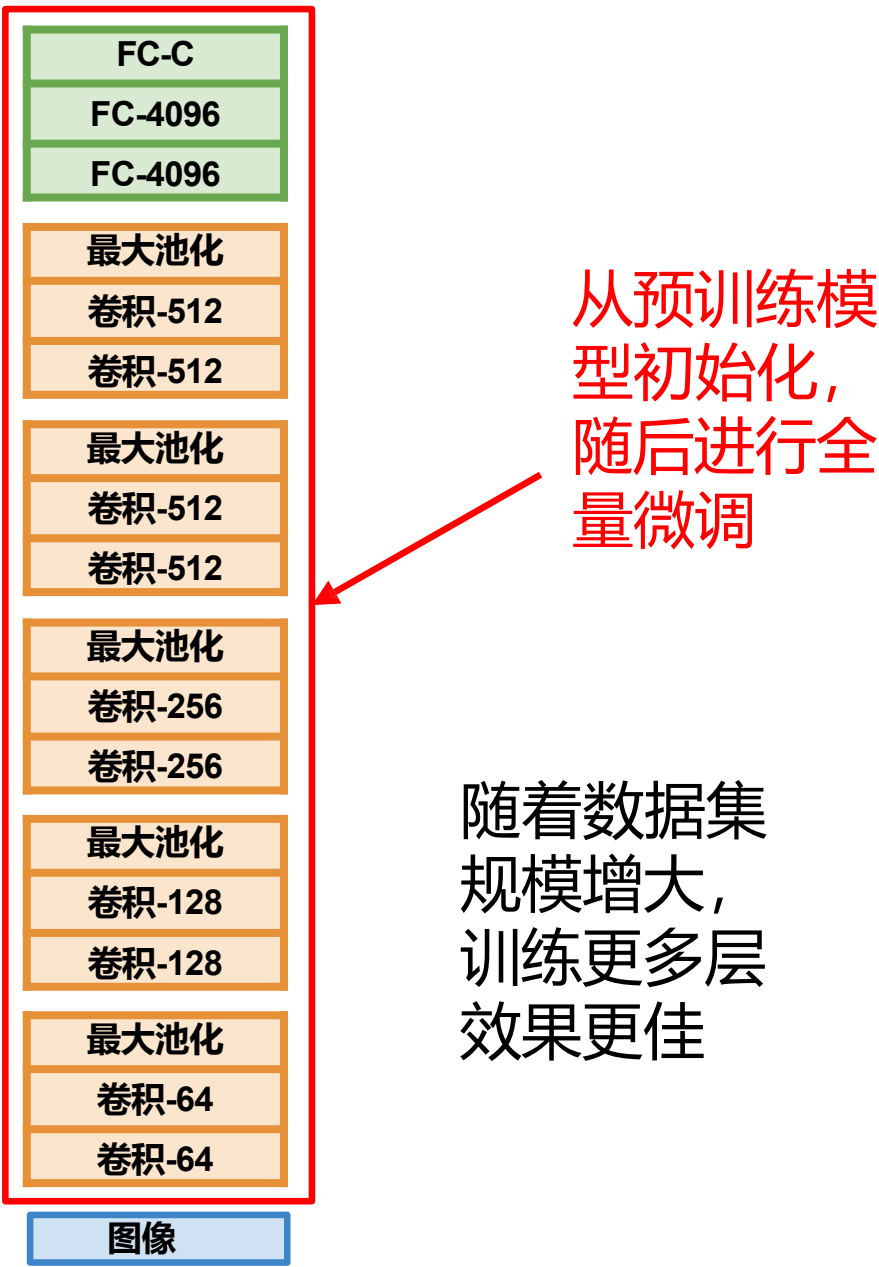
1. 在Imagenet上训练

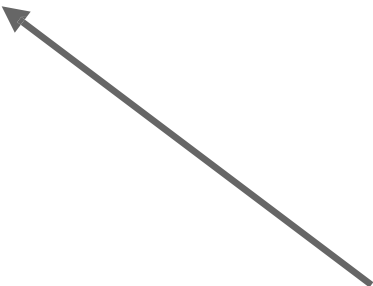


2. 小型数据集 (C个类别)



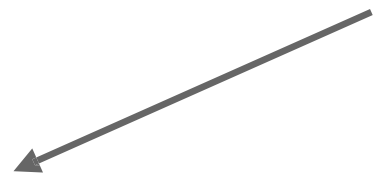
3. 更大数据集



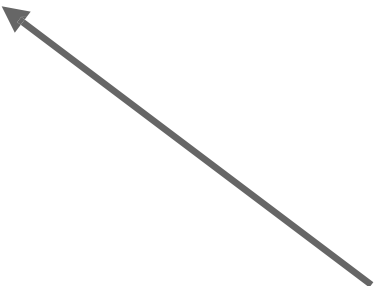


更具体

更通用

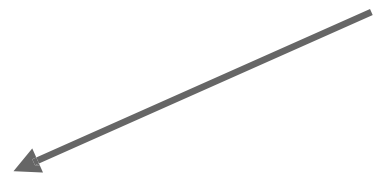


	非常相似的数据集	差异显著的数据集
数据极少	?	?
相当多的数据	?	?



更具体

更通用



	非常相似的数据集	差异显著的数据集
数据极少	在最终层使用线性分类器	?
数据量相当大	对模型所有层进行微调	?



更具体

更通用

	非常相似的数据集	差异显著的数据集
数据极少	在最终层使用线性分类器	尝试其他模型或收集更多数据
数据量相当大	微调模型所有层	要么对模型所有层进行微调，要么从头开始训练！

项目及其他场景的要点：

手头有感兴趣的数据集，但图像数量少于100万张？

1. 寻找包含相似数据的超大规模数据集，在此基础上训练大型模型
2. 将学习成果迁移至目标数据集

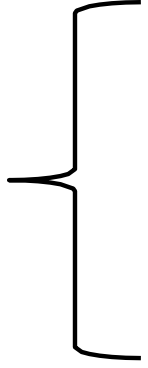
深度学习框架提供了一个预训练模型的“模型动物园”，因此您无需自行训练模型

PyTorch: <https://github.com/pytorch/vision>

Huggingface: <https://github.com/huggingface/pytorch-image-models>

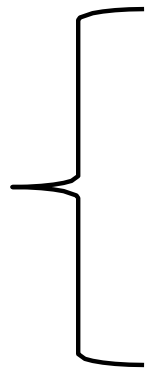
两大主题

如何构建卷积神经网络?



卷积神经网络中的层结构
激活函数
卷积神经网络架构
权重初始化

如何训练卷积神经网络?



数据预处理
数据增强
迁移学习
超参数选择

超参数选择

步骤1： 检查初始损失

步骤2： 对小样本进行过拟合

步骤3： 寻找使损失下降的学习率

使用上一步的架构，使用全部训练数据，开启小幅权重衰减，寻找能在约100次迭代内使损失显著下降的学习率

推荐尝试的学习率： $1e-1$, $1e-2$, $1e-3$, $1e-4$, $1e-5$

超参数选择

- 步骤1：** 检查初始损失
- 步骤2：** 对小样本进行过拟合
- 步骤3：** 寻找使损失下降的学习率
- 步骤4：** 超参数粗网格搜索，训练约1-5个 epoch
- 步骤5：** 优化网格，延长训练时间
- 步骤6：** 观察损失曲线与准确率曲线（后续幻灯片）

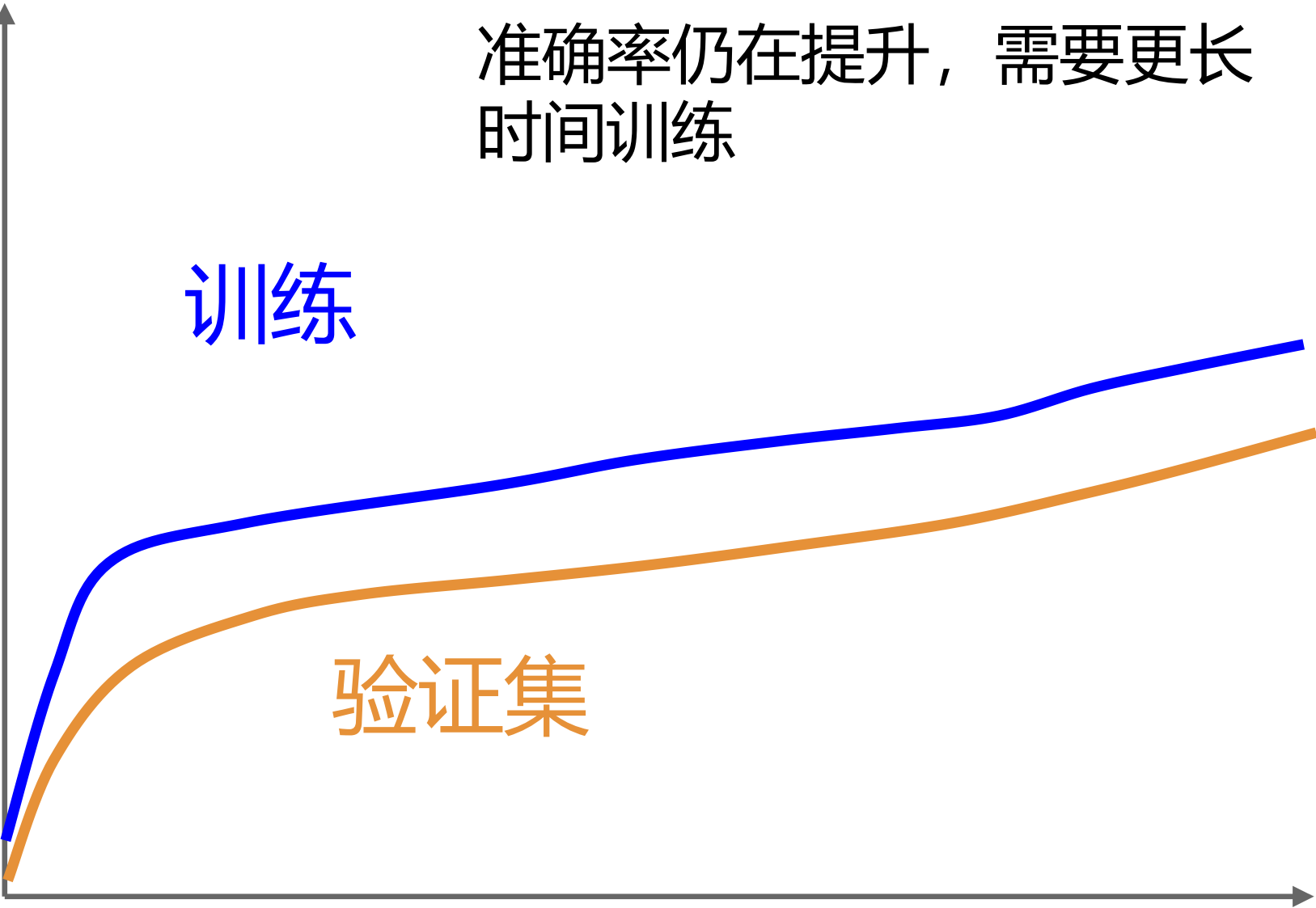
准确率

准确率仍在提升，需要更长时间训练

训练

验证集

时间



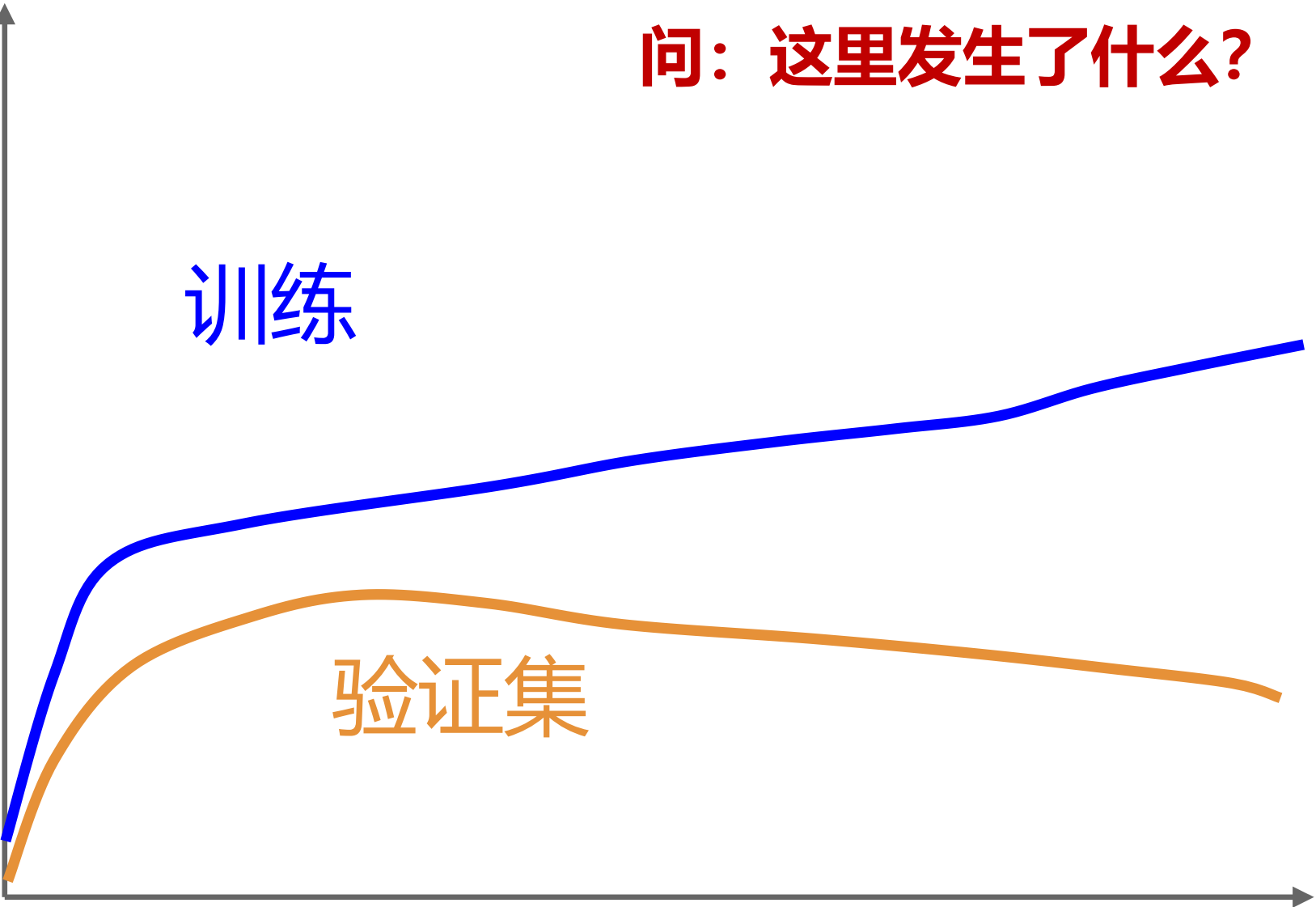
准确率

问：这里发生了什么？

训练

验证集

时间



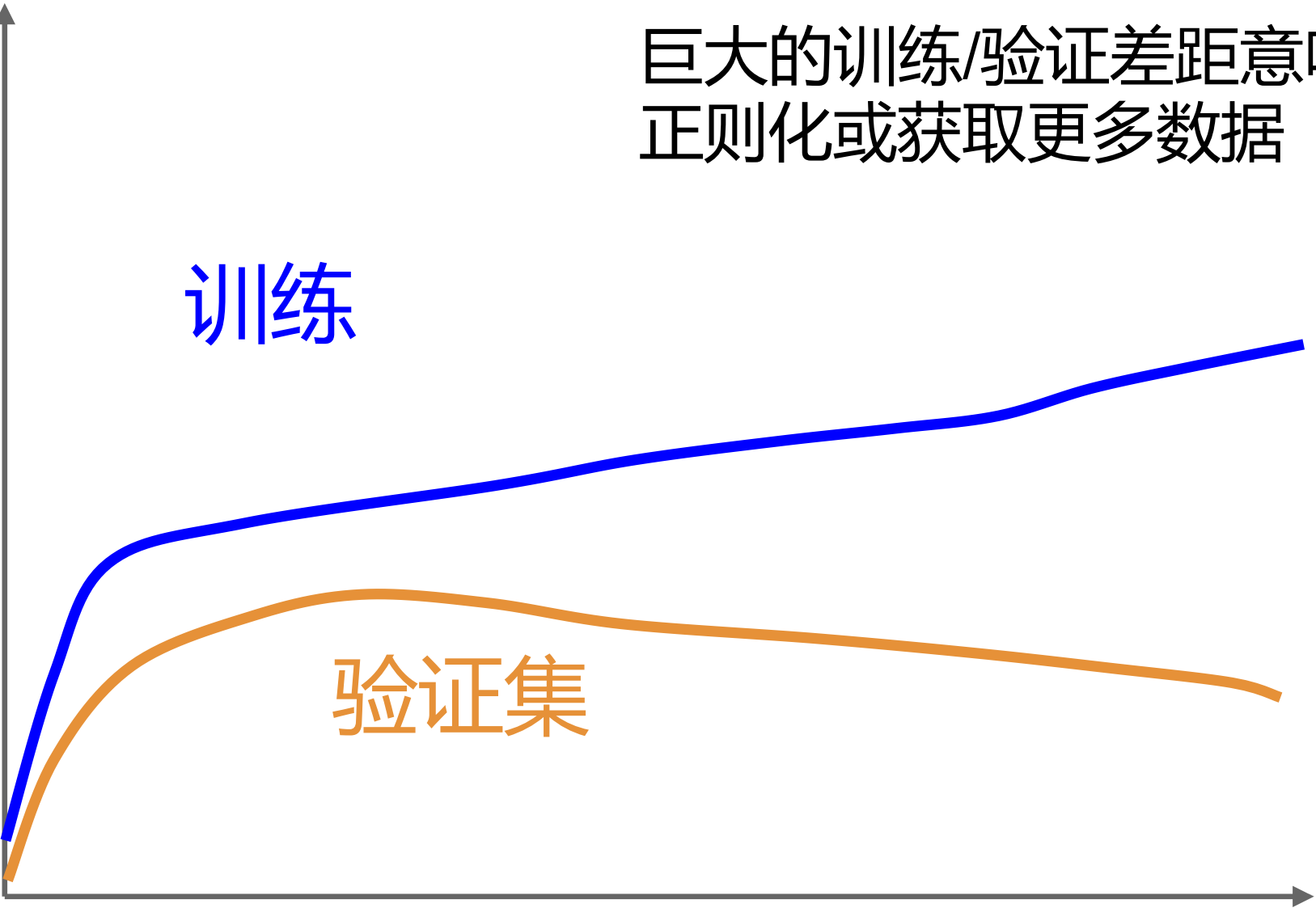
准确率

巨大的训练/验证差距意味着过拟合！增加正则化或获取更多数据

训练

验证集

时间



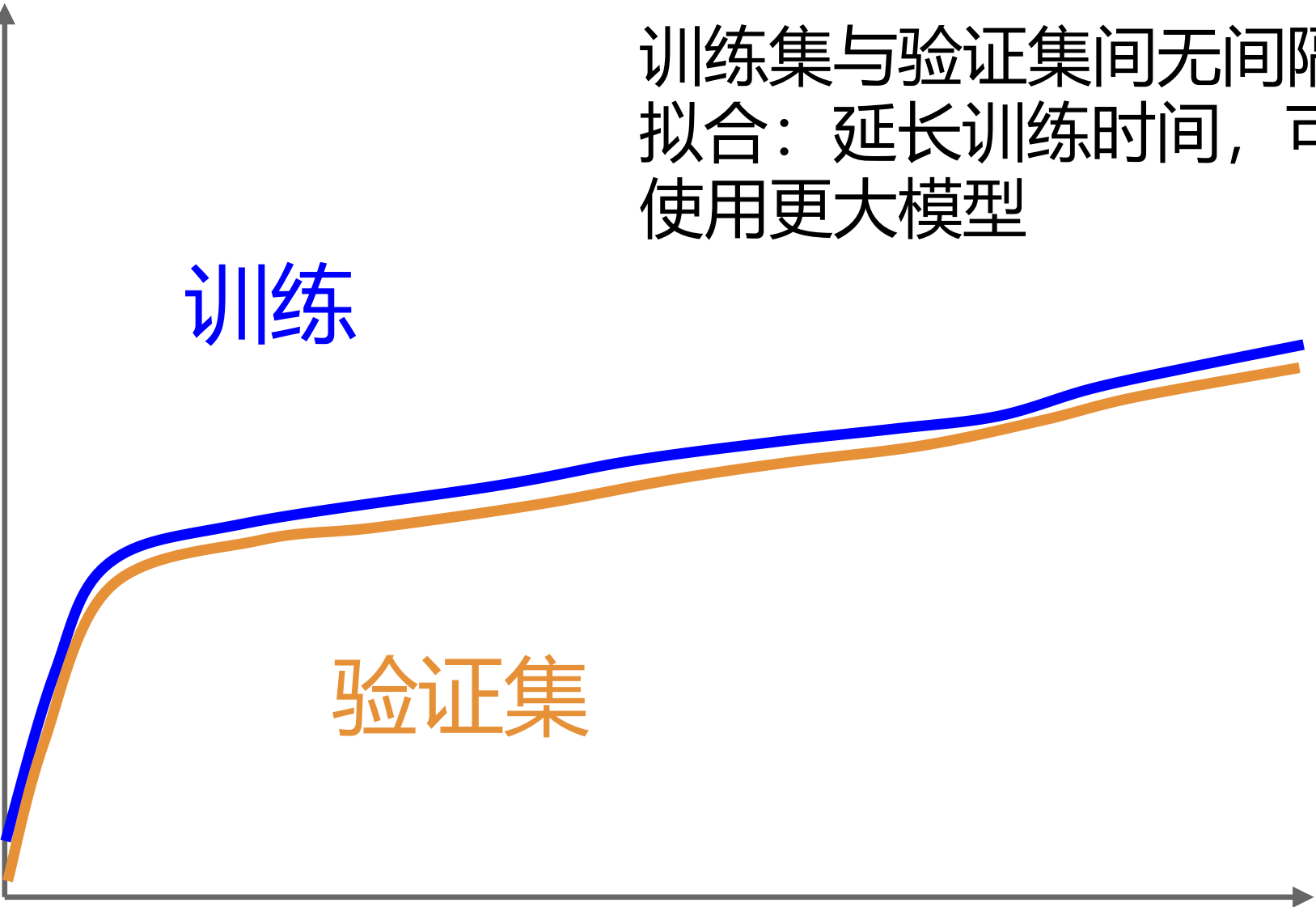
准确率

训练集与验证集间无间隔表示欠拟合：延长训练时间，可能需要使用更大模型

训练

验证集

时间



超参数选择

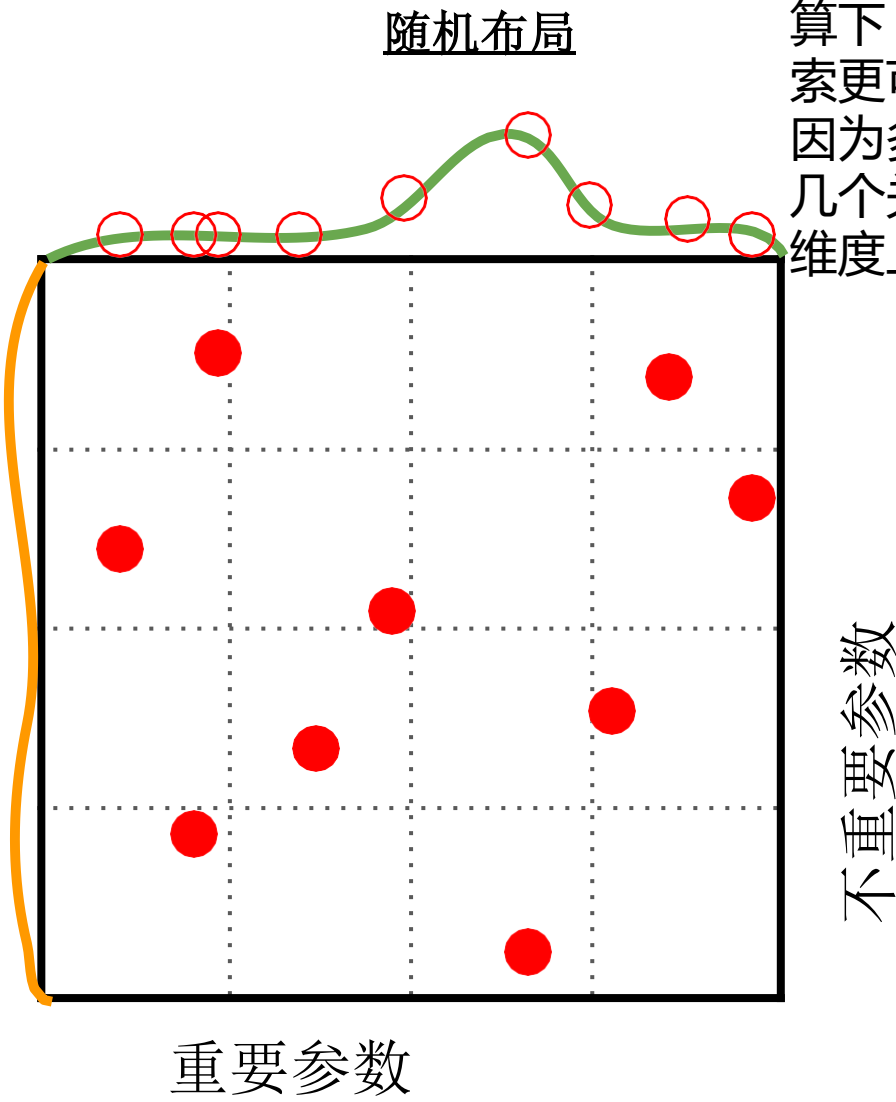
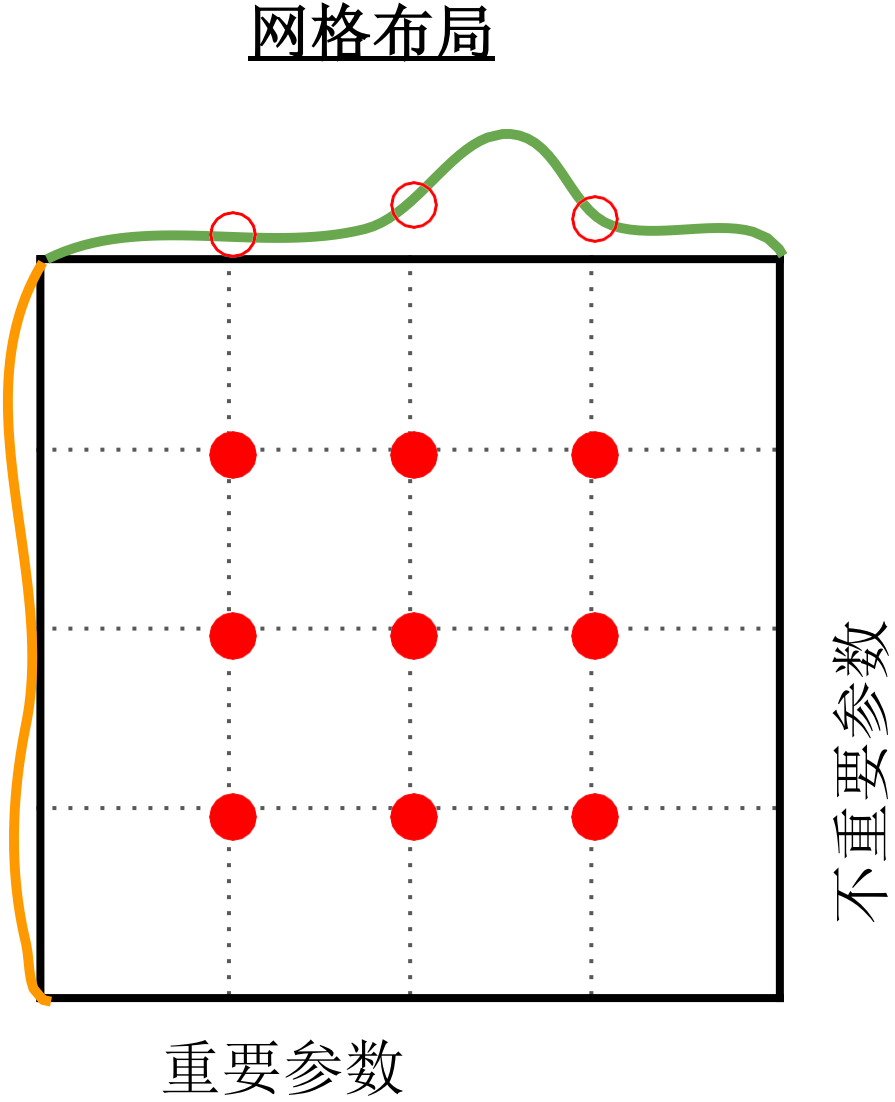
- 步骤1：检查初始损失
- 步骤2：对小样本进行过拟合
- 步骤3：寻找使损失下降的逻辑回归**模型**
- 步骤4：粗网格训练约1-5个 **epoch**
- 步骤5：精细网格训练更长时间
- 步骤6：观察损失曲线和准确率曲线
- 步骤7：**转至步骤 5**

随机搜索与网格搜索

网格搜索把每个超参数的可能取值等间距地划分，形成一个“网格”。然后在这些格点上穷举所有组合。

若某些参数（如横轴“重要参数”）对性能影响大，而另一些参数（如纵轴“不重要参数”）影响小，则：

- 网格搜索会浪费大量计算资源在“不重要的维度”上。
- 即使你加密网格，也只是在无关紧要的方向上细化。



随机搜索不是穷举，而是随机地采样参数组合。每次从所有参数空间中随机选择一组取值。

红点位置随机分布，因此：有更高的概率覆盖重要参数方向的多种情况；不会在不重要的维度上过度采样。

优势：论文证明：在相同的计算预算下（相同次数的实验），随机搜索更可能找到更优的超参数组合。因为多数模型性能主要依赖于少数几个关键参数，随机搜索能在这些维度上探索得更充分。

伯格斯特拉等人（2012）的插图由谢恩·朗
普雷绘制，版权归属CS231n 2017

绿色线条：代表模型性能随“重要参数”的变化。
橙色线条：表示“不重要参数”对性能影响不大（几乎是平的）。
圈出的红点：实验采样点（每次训练一个模型）。

摘要

我们从宏观层面回顾了8个主题：

1. 卷积神经网络中的层（卷积层、全连接层、归一化层、Dropout层）
2. 神经网络中的激活函数（ReLU、GELU等）
3. 卷积神经网络架构（VGG、ResNets）
4. 权重初始化（保持激活分布）
5. 数据预处理（减去均值，除以标准差）
6. 数据增强（裁剪、抖动）
7. 迁移学习（先在ImageNet上训练）
8. 超参数优化（损失函数监测+随机搜索）